MFDStudio - Detailed User Guide

design in `mfd_editor`, C++17 API generation, live client integration, framebuffer plugin, and launch scripts

Document generated on 2026-05-10 12:51
Scope: editor, architecture, generator, client API, framebuffer plugin, launch scripts
Target audience: MFD authors, C++ integrators, and runtime teams

Editor · ch. 1-4	API · ch. 5-6	Plugin · ch. 7

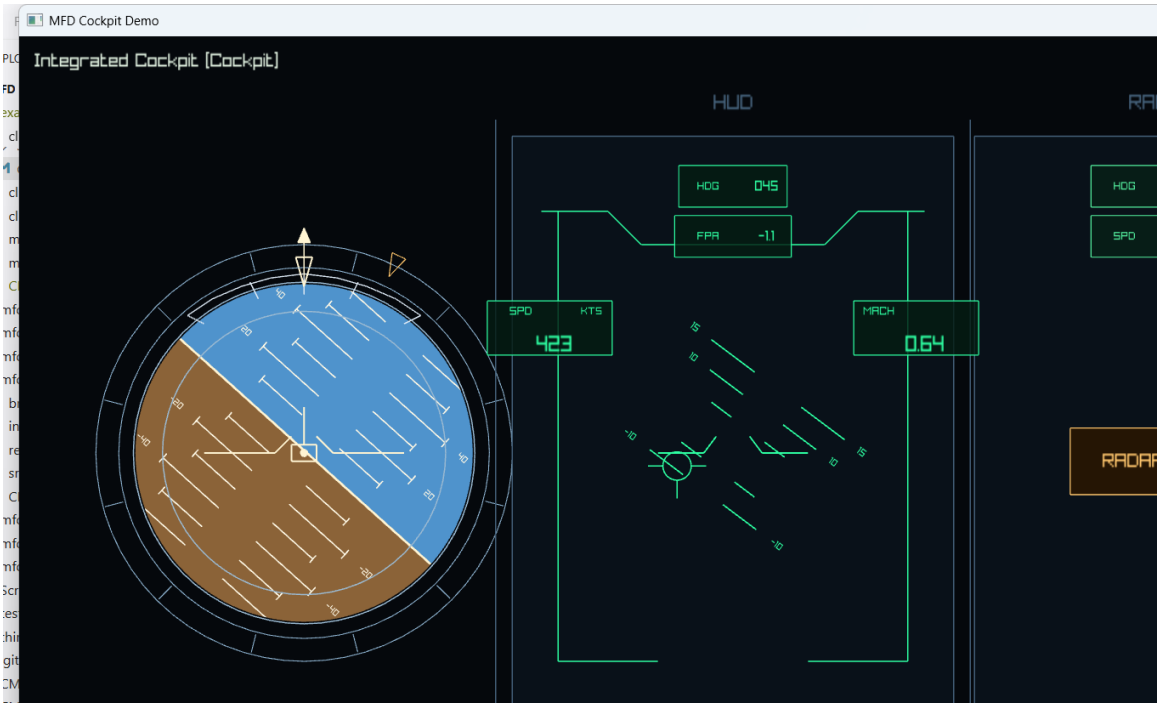
Target document:

- MFD page and window authors
- C++ integrators who must control a window live
- teams who want to capture the framebuffer via a DLL plugin

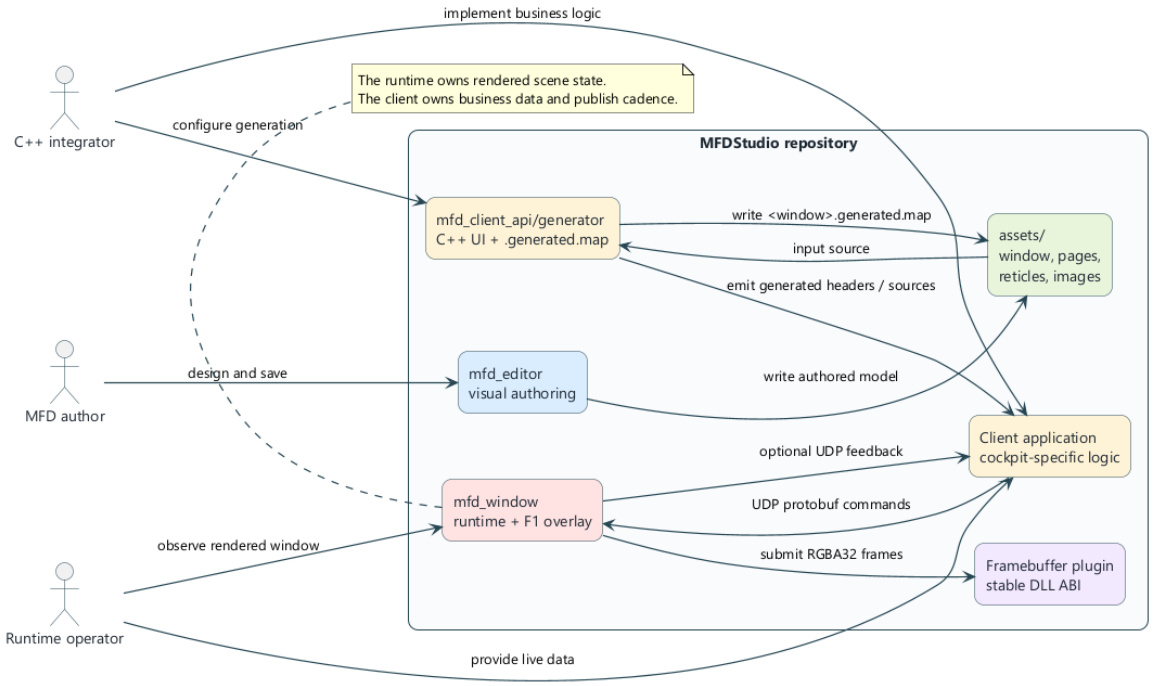
Table of Contents

The table of contents will be updated by Word.

1. What is MFDStudio



Cockpit runtime screenshot



MFDStudio Overview

MFDStudio is a C++17 / CMake toolbox for designing, executing and controlling MFD type 2D windows from JSON files. The repository voluntarily separates three responsibilities:

- the content author defines versioned assets: window, pages, reticles, images, fonts
- the runtime `mfd_window` loads these assets and renders the active page
- the external client sends live commands via UDP to modify the runtime state without rewriting the assets

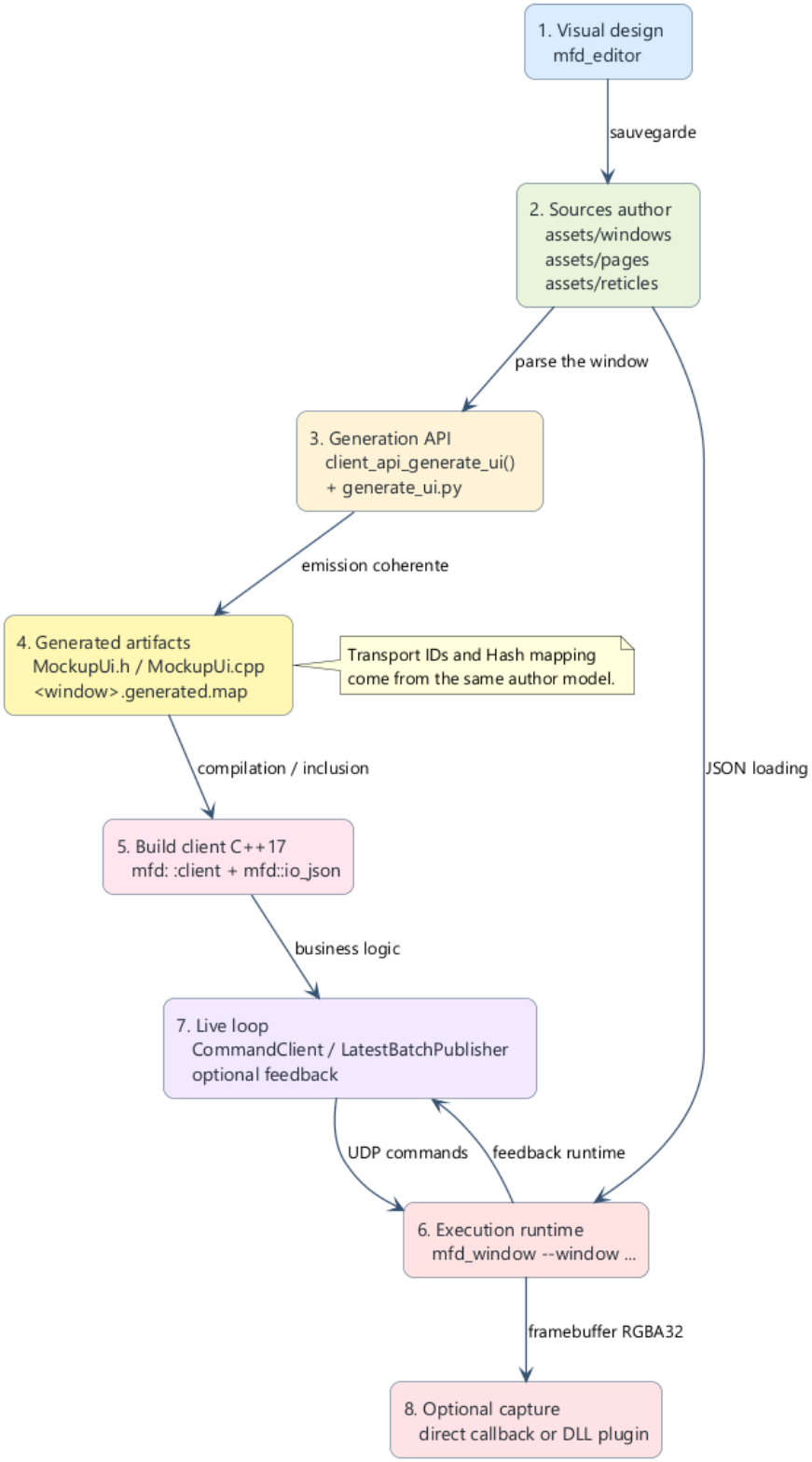
The historical technical prefix of the repository remains `mfd` in namespaces, CMake targets, folders, and public APIs. You must therefore distinguish:

- the product name: `MFDStudio`
- the technical prefix: `mfd`

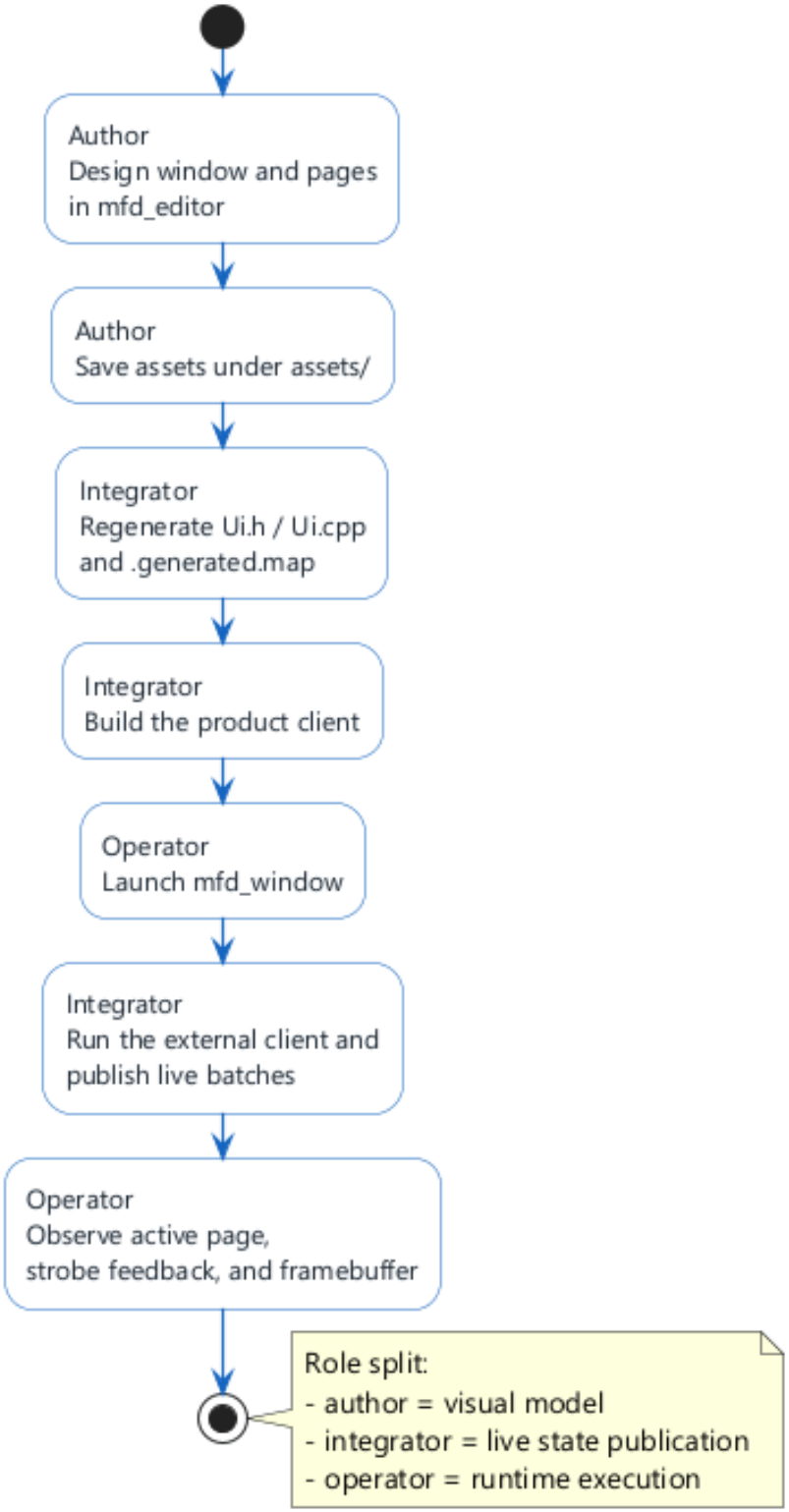
1.1 The main components

Component	Role
mfd_editor	visual tool for authoring windows, pages, and reticles
assets/	versioned source of truth
mfd_client_api/generator	generates typed C++ wrappers and the companion .generated.map file
mfd_window	runtime host that opens a JSON window, renders the active page, and accepts live commands
client_mockup	reference client that shows how to use the public API
mfd_window_plugin_api	stable C ABI for framebuffer capture DLL plugins

1.2 The minimum mental model



Pipeline authoring to runtime



End-to-end workflow for an author, integrator, and runtime operator

The normal flow of an MFDStudio project is as follows:

1. 1. we author a window and its pages in `assets/` with `mfd_editor`
2. 2. we generate a C++ client API specific to this window
3. 3. we compile a business client which uses this generated API
4. 4. we launch `mfd_window` on the JSON window
5. 5. the client sends UDP commands to the runtime
6. 6. the runtime can send UDP feedback to the client

7. 7. optional, a DLL plugin receives the RGBA32 framebuffer at each frame

1.3 Who does MFDStudio serve?

MFDStudio is suitable when you need:

- cleanly separate graphic authoring and real-time logic
- keep a readable and versionable author model
- drive pages by an external client without embedding the rendering engine in the client
- generate C++ type wrappers from an author window
- plug a framebuffer capture pipeline into a stable DLL ABI

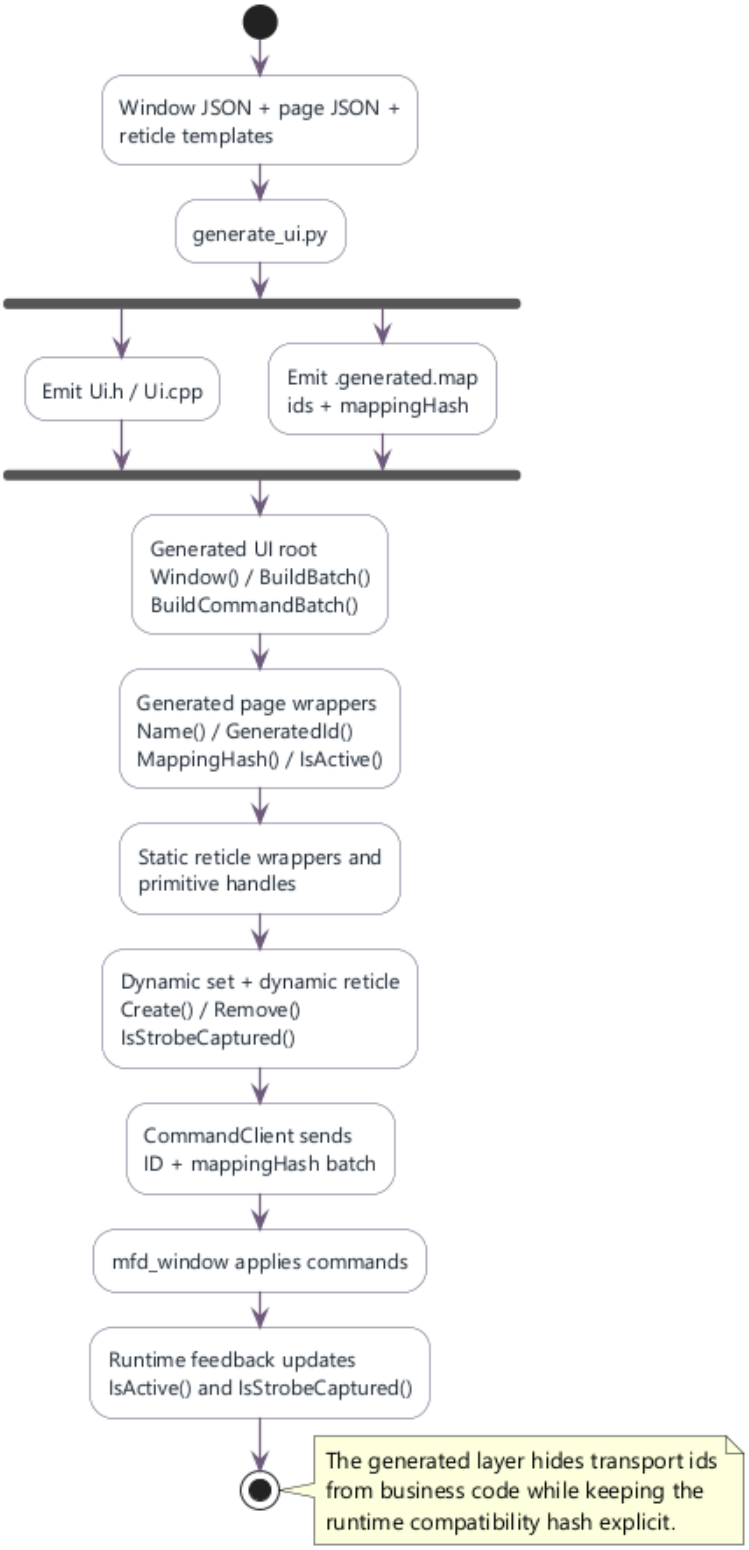
1.4 What MFDStudio is not

MFDStudio is not:

- a general Qt-type UI framework
- a 3D scene authoring system
- a generic network protocol independent of assets
- a plugin system open to any C++ ABI

⚠ WARNING	The runtime remains the owner of the scene state and the rendering. The client remains the owner of the business data and the transmission rate. It is this separation that gives stability to architecture.

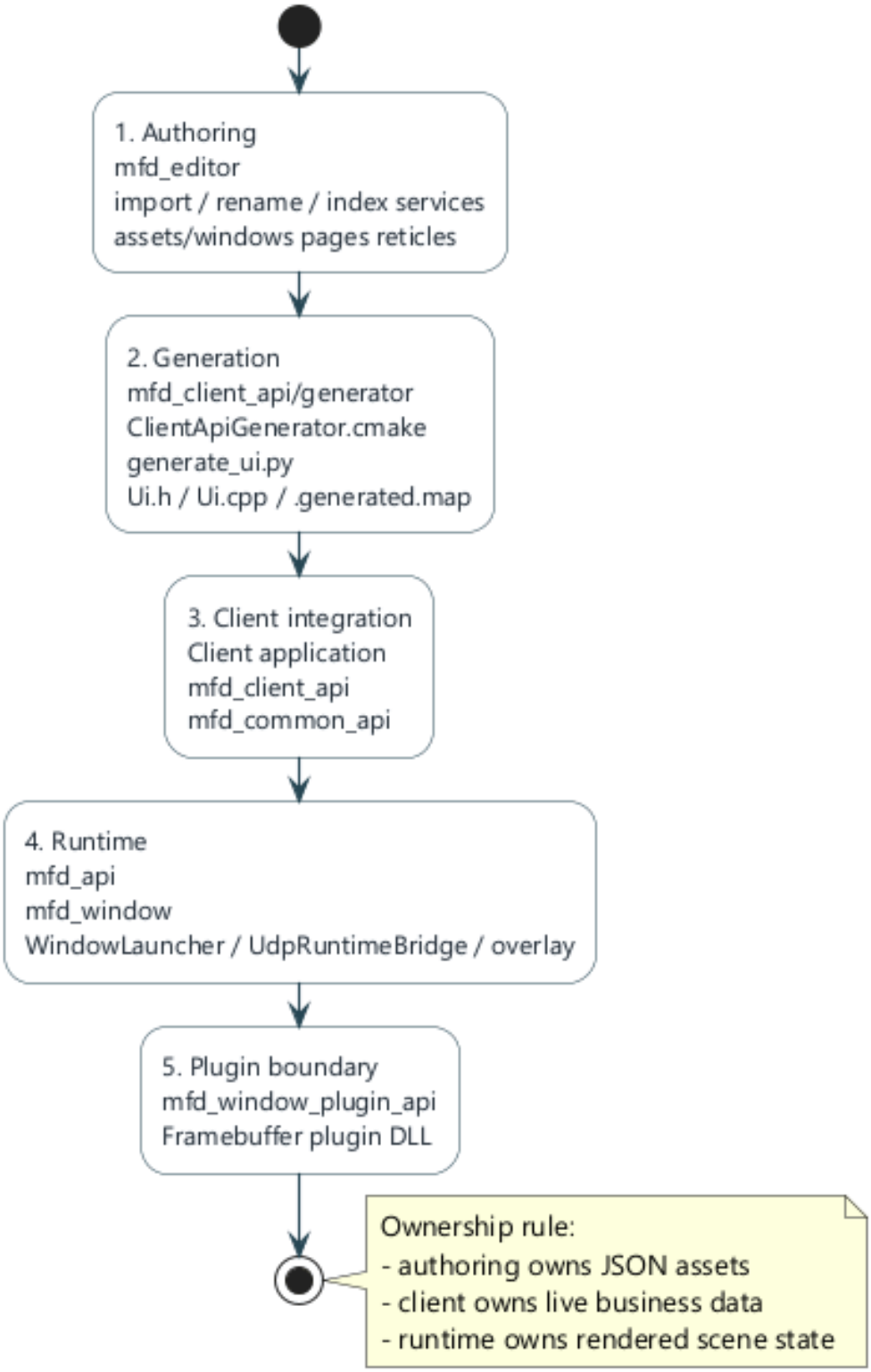
2. Detailed Architecture



API and transport model

The repository architecture is modular. Each module keeps a deliberately narrow responsibility.

2.1 Depot modules



Modular layers of the repository

Module	Primary Responsibility
mfd_common_api	common types, low-level authoring model, transport and commands

mfd_api	loading JSON, runtime, rendering, scene registry
mfd_client_api	client overlay: handles, batching, publication, feedback
mfd_client_api/generator	derives a C++ UI and a transport map from a JSON window
mfd_editor	visual authoring and protected workflows on assets
mfd_window	generic runtime launcher and debug overlay
mfd_window_plugin_api	Stable C contract for framebuffer capture plugins
examples/	reference clients and plugins

2.2 Author / runtime / client separation

The architecture is based on three layers which must not be mixed:

Layer	Possessed	Must not own
Authoring	visual structure and JSON assets	live business logic
Runtime	rendering and state authoritarian scene	knowledge of the customer business domain
Customer	live data, cadence, control intentions	the window rendering engine

2.3 The role of the file .generated.map

The generator emits two coherent artifacts:

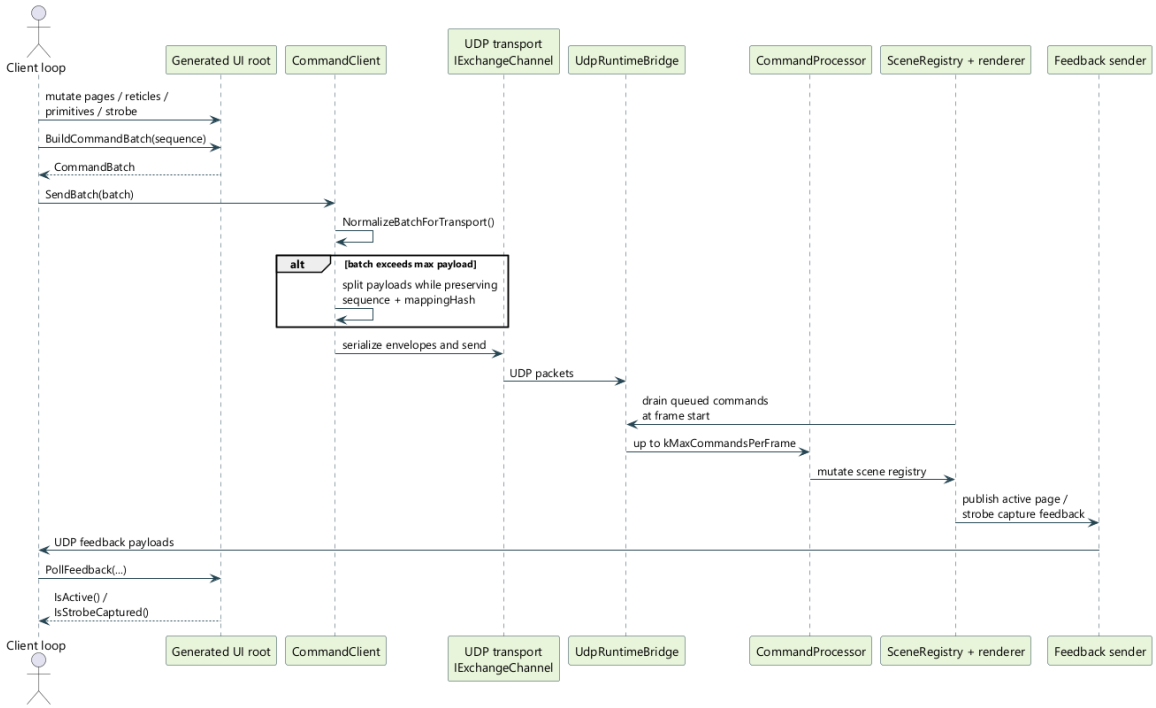
- a C++ header/source generated for the client
- a `<window>.generated.map` file

The `.generated.map` contains stable transport IDs for:

- the pages
- static reticles
- the primitives exposed
- dynamic templates
- the blink types

It also contains a `mappingHash` used to prove that the client and the window are talking about the same author model.

2.4 Runtime loop and feedback



Command sequence and feedback

In the recommended mode:

- a UDP worker receives command packets
- the render thread drains commands at the start of the frame
- the runtime applies the mutations via the `CommandProcessor` and the scene
- the runtime can send authoritative feedback to the client

This feedback is used in particular to:

- the active page rendered
- the resolved state of the strobe
- capturing a dynamic reticle by the strobe

2.5 Author model

The minimal author model is:

- a `window`
- a list of `pages`
- of the `staticReticles` instances on each page
- a folder of reusable reticle templates
- of the `dynamicReticleBindings` on the pages
- A `strobe` optional per page
- of the `blinkTypes` defined at page level

The logical author and client coordinates are normalized in `[-1, 1]`.

2.6 Practical consequences for integration

For a clean integration:

- authoring the visual structure into JSON
- only expose the necessary primitives to the client
- use the generated API rather than text names everywhere
- preserve `CommandClient` as final sending layer
- reserve framebuffer plugins for image output, not for business control

3. Prerequisites and tools to build

3.1 Recommended Toolchain

The target deposit in practice:

- Visual Studio 2022
- CMake 3.25 or higher
- C++17
- Python 3 for the generator

3.2 Build local recommends

The preferred local flow is Debug Win32:

CODE POWERSHELL

```
cmake --preset vs2022-win32
cmake --build --preset debug-win32
ctest --preset test-debug-win32
```

3.3 Useful binaries as needed

To author and test the main flow:

CODE POWERSHELL

```
cmake --preset vs2022-win32
cmake --build --preset debug-win32 --target mfd_window mfd_framebuffer_stdout_plugin
client_mockup mfd_editor
```

For one specific generated client:

CODE POWERSHELL

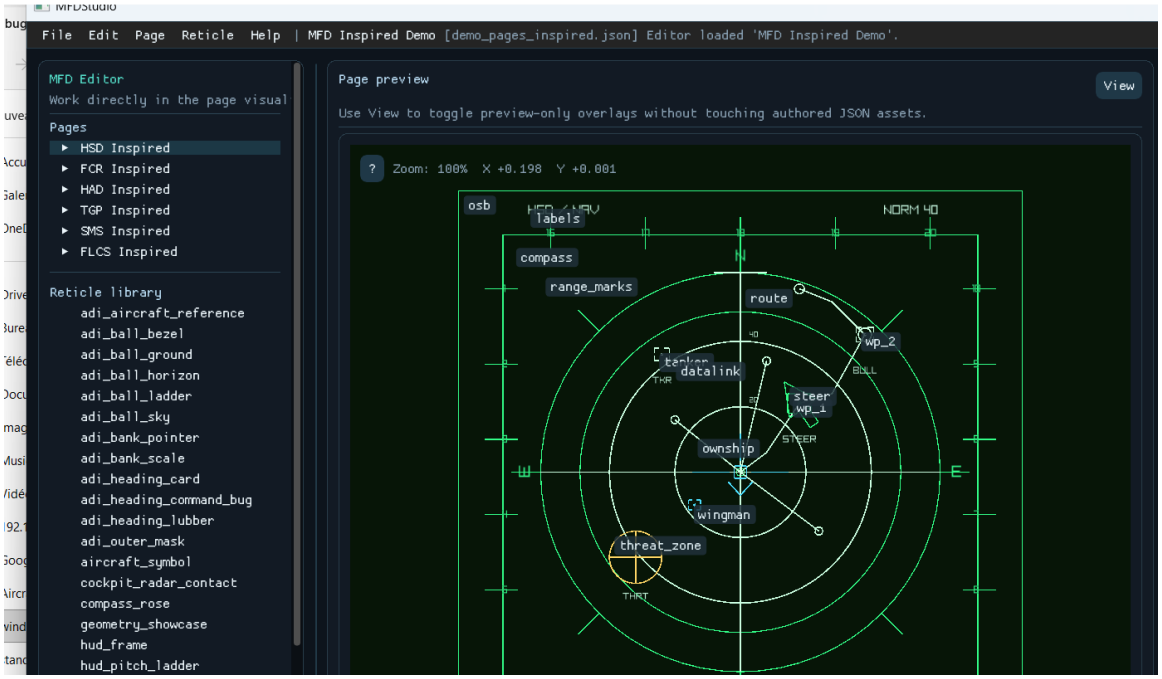
```
cmake --build --preset debug-win32 --target client_mockup_minimal
```

3.4 Tree structure to know

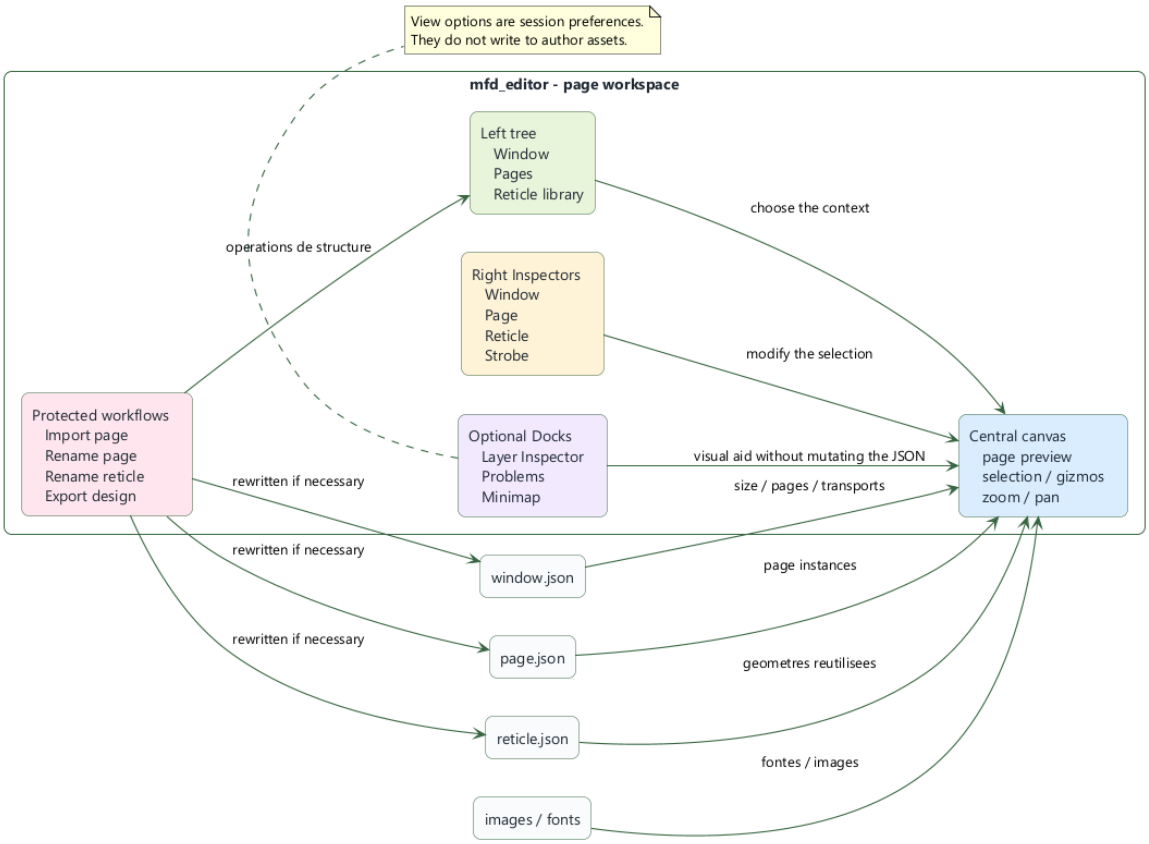
Case	Expected content
assets/windows	JSON root windows
assets/pages	JSON pages
assets/reticles	reticle templates
assets/fonts	possible fonts
examples	reference clients and plugins
Scripts	launch batches
_Exec	runtime staging regenerated by the build

✦ TIP	Author in `assets/` and not in `_Exec`. `_Exec` is a runtime staging area, not the long-term source of truth.

4. Use mfd_editor to design a window and its pages



`mfd_editor` screenshot



Editor workspace

This section describes how to use `mfd_editor` in detail to author a window, its pages, its reticles, and the associated asset-maintenance workflows.

4.1 Launch the editor

Once `mfd_editor` is built:

- launch `mfd_editor.exe`
- or launch the staged executable under `_Exec`

At startup, the editor does not automatically reload a copy from `_Exec`. It opens an empty document until the user:

- load an existing window
- or create a new window

4.2 Anatomy of the interface

The editor is organized around four areas:

Area	Use
left shaft	window, pages, reticle library
central canvas	page preview, selection, gizmos, zoom/pan
straight inspectors	contextual window editing, page, reticle, strobe
optional docks	Layer Inspector, Problems, Minimap

The preview remains the main surface. The Dockes panels reserve a real space and must not hide the canvas.

4.3 Create a new window

Menu:

- `File > New window from scratch`

The popup allows you to enter:

Field	Sense
Window file	path of the JSON window to create
Window title	title visible from window
Size (px)	window width/height
Position (px)	initial screen position
Font file (optional)	font used for text rendering
Reticle library folder	reusable templates folder

These fields correspond directly to the root of the JSON window.

4.4 Configure UDP transports

The editor exposes two configuration groups.

Incoming commands to window

Option	Effect
Enable command UDP	activates the command entry point
Command address	IPv4 listening or link address
Command port	UDP port
Command max packet	max size of a datagram

Feedback coming out of runtime to clients

Option	Effect
Enable feedback UDP	activates the return flow
Feedback address	target IPv4 address
Feedback port	UDP feedback port
Feedback max packet	max datagram size

For a single position, use `127.0.0.1`.

4.5 Create a first page

During window creation, you can activate `Create one initial page` then enter:

Field	Sense
First page name	public page name
First page title	human title
First page file	JSON page path
First page background	background color

This first page becomes the `defaultPage`.

4.6 Save what the editor writes

The document first exists in memory. Nothing persists until the user does:

- `File > Save`

The writing may concern:

- the JSON window
- the JSON of the initial page
- reticle templates if the workflow has created them

4.7 Add, choose and organize pages

To add a page:

- `Page > New page`

To choose the default page:

- select the page in the tree
- enable `Default page for this window` in the page inspector

4.8 Design of a page: layers, static reticles, dynamic bindings, strobe

Page layers

Each page declares `layers` orders. They are the runtime source of truth for the drawing order.

Static reticles

A page can contain `staticReticles` :

- either library template instances
- either reticles inline

Each instance aims at `layerId`.

Dynamic reticle bindings

Section `Dynamic reticles` in the page inspector:

8. 1. choose one `Reticle template`
9. 2. choose one `Layer`
10. 3. to click `Add`

Each entry creates a runtime binding, not a static instance drawn by default. The live client will then create the real runtime instances.

Page strobe

A page can expose a `strobe` optional. The editor allows you to assign a strobe template, its initial position and its capture/magnetization configuration.

4.9 Important page fields

The JSON page supports in particular:

Canonical field	Use
name	public page name
title	human title
backgroundColor	bottom
layers	ordered runtime layers
dynamicReticleBindings	dynamic templates allowed on the page
blinkTypes	page blink types
defaultBlink	blink by default
view.center	center of view
view.zoom	zoom
staticReticles	static reticles
strobe	optional strobe

4.10 Supported primitives for constructing reticles

The families of primitives supported by the author model are:

Primitive	Typical usage	Specific fields
text	labels, values	text, fontSize, letterSpacing
time	clock	format, utc, fontSize, letterSpacing
line	features	start, end
circle	markers	radius
ring	crowns	innerRadius, outerRadius, segments
rectangle	panels	width, height, size
ellipse	oval markers	width, height, radiusX, radiusY
square	square markers	size, width, height

diamond	waypoints	size, width, height
triangle	pointers	points
polyline	roads, frames	points, closed
bezier	curves	controlPoints, segments
arc	sectors, arcs	radius, startAngleDegrees, endAngleDegrees, segments
image	bitmaps	file, width, height, size

Common primitive fields cover:

- `position`
- `rotationDegrees`
- `scale`
- `visible`
- `stroke`
- `thickness`
- `lineStyle`
- `filled`
- `fill`

4.11 Selection and direct editing on the canvas

The page preview supports:

- `Ctrl+click` to add or remove a reticle from the selection
- `Esc` to empty the selection
- dragging a selection to move a group
- `Ctrl+C`, `Ctrl+X`, `Ctrl+V`, `Del`

In case of superimposed reticles:

11. 1. right click in the area
12. 2. choice of the desired reticle in the context menu
13. 3. copy/cut/delete operations on the current selection

4.12 Menu View from the preview

The available toggles are:

Option View	Effect
Layer Inspector	layer dock and focus by layer
Minimap	mini navigation view

Problems	dockes diagnostics
Highlight reticle usages	highlights the pages that reference the selected template
Reticle names	displays names
Gizmos	shows visual manipulations
Page context	preview context linked to the page

These options are session preferences. They do not rewrite the author JSON.

Fullscreen preview

The button `[]`, `F11` Or `Esc` allows you to:

- hide sidebar, inspectors and docks
- keep the canvas interactive
- restore the previous state then

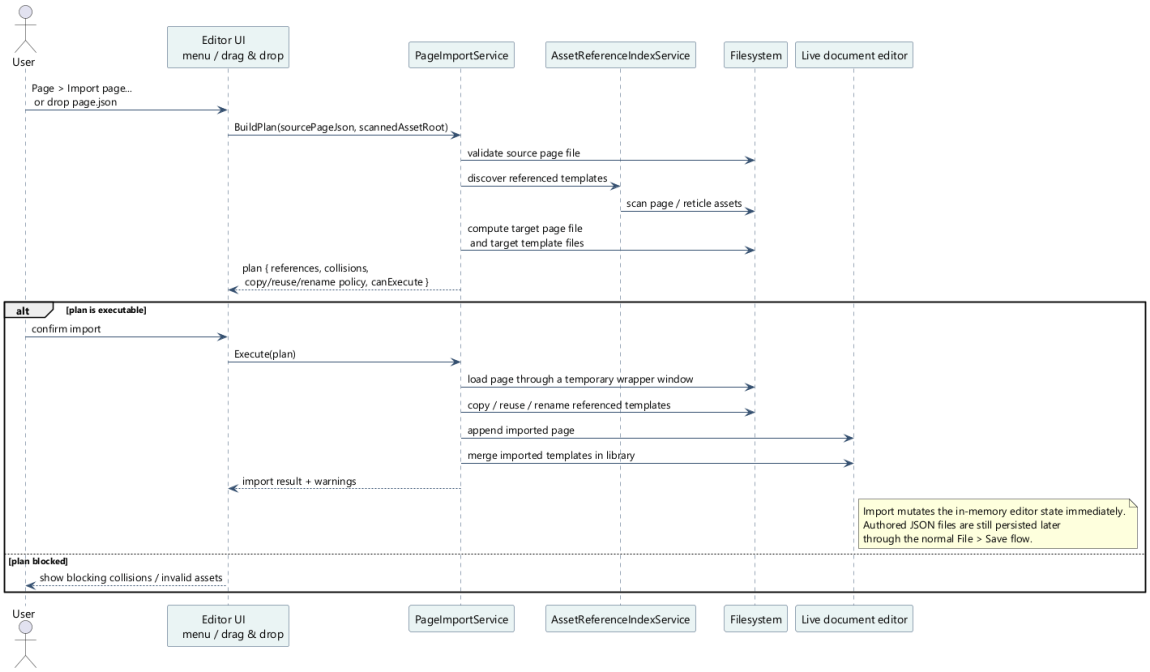
4.13 Layer Focus Mode

THE `Layer Inspector` allow :

- duty `Full View`
- to list the runtime layers
- to display a thumbnail and a counter per layer
- to focus a layer to leave only its reticles editable

The other visible layers remain rendered, but dimmed.

4.14 Import an existing page



Page import sequence in the editor

Entries:

- `Page > Import page...`
- or drag and drop a page JSON

The workflow calculates a plan before execution:

- source page
- target in current tree
- template references
- collisions
- copy/reuse/rename strategy

The current rules are:

- target file missing: copy
- identical file already present: reuse
- different file already present: creation of a renamed copy

4.15 Delete or detach a page

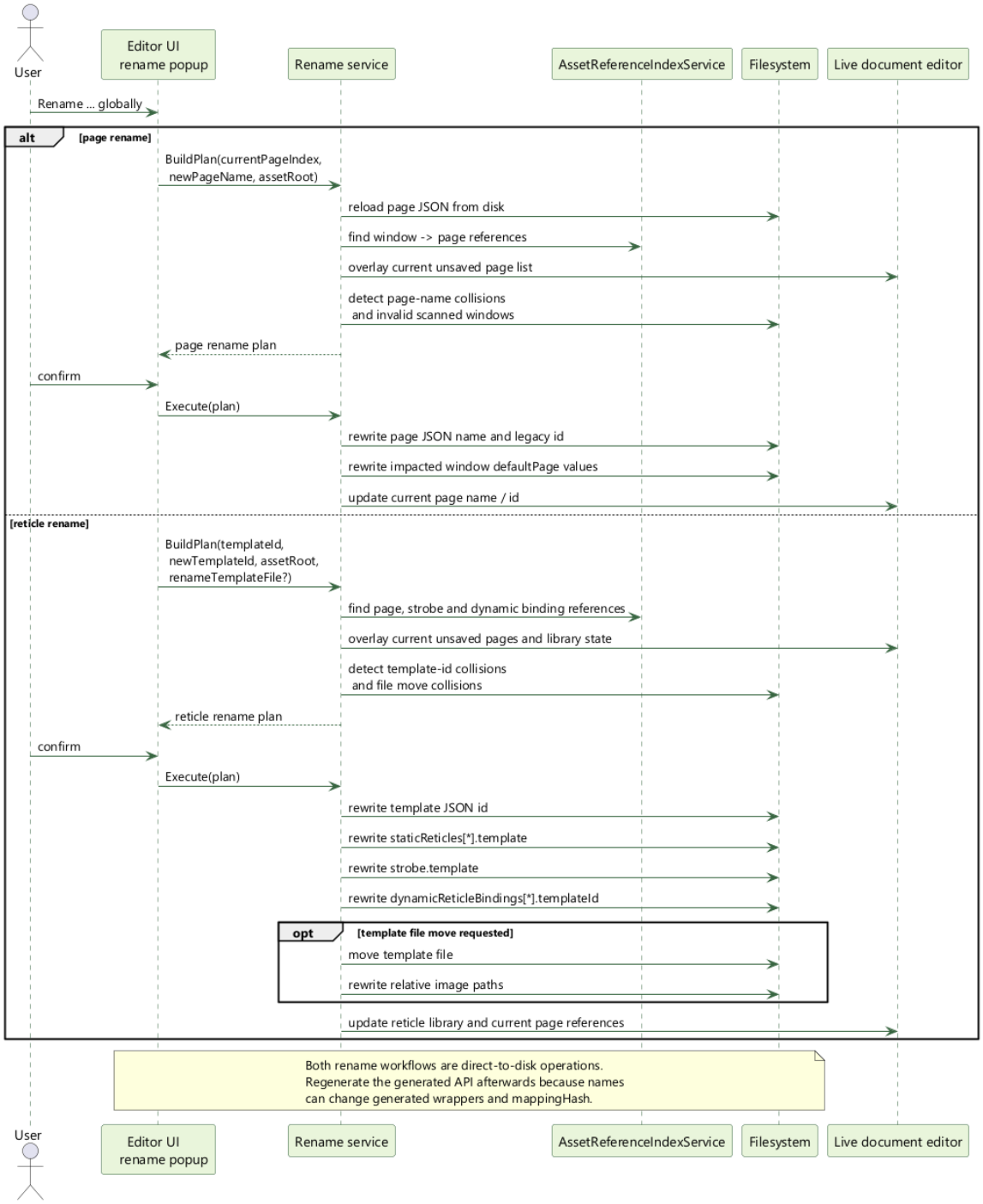
Two distinct actions exist:

Action	Effect
Remove page from window	removes the page from the current window but keeps the file
Delete page asset...	removes the page and marks its JSON for deletion

Security guards:

- the window must keep at least one page
- we do not delete the default page without choosing another one
- deletion out `assets/` blocked without explicit override

4.16 Properly rename a shared page



Global page/reticle rename sequence

Entries:

- right click page `Rename page globally...`
- inspector button
- menu `Page > Rename current page globally...`

The rename service:

- rescan references `window -> page`
- rewrites the page JSON
- rewrite the `defaultPage` impacts
- blocks collisions before any mutation
- warns that the generated API and the `mappingHash` can change

4.17 Properly renaming a shared reticle template

Entries:

- right click template `Rename reticle globally...`
- inspector button
- menu `Reticle > Rename selected library reticle globally...`

The workflow:

- rewrite the `id` of the template
- rewritten `staticReticles[*].template`
- rewritten `strobe.template`
- rewritten `dynamicReticleBindings[*].templateId`
- can also rename the template file
- updates relative image paths if file moves

4.18 Highlight use of a template

Normal flow:

14. 1. select a reticle template in the tree
15. 2. enable `View > Highlight reticle usages`
16. 3. the editor highlights:

- the pages that reference this template
- the corresponding instances on the active page

4.19 Export of design documentation

Menu:

- `File > Export > Export design...`

The exported package may contain:

Exit	Content
README.md	package entry exports
window_icd.md	window view
pages/*.md	detailed views per page
images/*_design_exploded.png	exploded views if rendering was successful

data/design_manifest.json	design manifesto exports

Options available in the popup:

- Markdown ICD snippets
- exploded views
- contact details
- strobe
- blink
- primitive ids
- hash mapping

4.20 Recommended authoring checklist

17. 1. create the window and its transports
18. 2. define the pages and `defaultPage`
19. 3. declare runtime layers
20. 4. compose static reticles and only expose useful primitives to the client
21. 5. declare the `dynamicReticleBindings` necessary
22. 6. set strobe if page needs it
23. 7. check `Problems`, `Layer Inspector`, `Minimap`
24. 8. save as `assets/`
25. 9. regenerate the API if the exposed runtime surface has changed

5. Generate the client API from the generator

For a C++ integrator, the generator is not a convenience wrapper. It is the compatibility boundary between authored JSON assets and the code that will publish live commands at runtime.

If generation is treated as a real build step, you get a typed, repeatable, window-specific integration contract. If generation is skipped, or if the generated files are stale, you lose the proof that the client and the runtime still agree on the same authored surface.

■ 5.1 Entry, exit, contract

The generator consumes one authored window entry point:

- one root window JSON
- every referenced page JSON
- every reticle template reachable from the window

It emits three integration artifacts:

- one generated C++ header
- one generated C++ source
- one `<window>.generated.map` sidecar

These outputs are one contract, not three independent files. The generated C++ layer bakes generated transport ids and the `mappingHash` into the typed wrappers. The `.generated.map` exposes the same canonical transport surface to the client, the runtime, and any raw-name tooling.

■ 5.2 Official CMake macro

The supported integration path is the `client_api_generate_ui(...)` CMake macro.

```cmake
client_api_generate_ui(
    WINDOW_JSON "assets/windows/demo_pages_cockpit.json"
    OUTPUT_HEADER "${CMAKE_CURRENT_SOURCE_DIR}/generated/MockupUi.h"
    OUTPUT_SOURCE "${CMAKE_CURRENT_SOURCE_DIR}/generated/MockupUi.cpp"
    OUTPUT_MAP "${MFD_ROOT_DIR}/assets/windows/demo_pages_cockpit.generated.map"
    NAMESPACE "mockup_ui"
    UI_CLASS_NAME "CockpitMockupUi"
    HEADER_INCLUDE "MockupUi.h")
```

Do not replace this macro with an ad hoc custom command unless you also reproduce its input-tracking behavior. The difficult part is not launching Python; the difficult part is ensuring CMake knows when authoring assets changed.

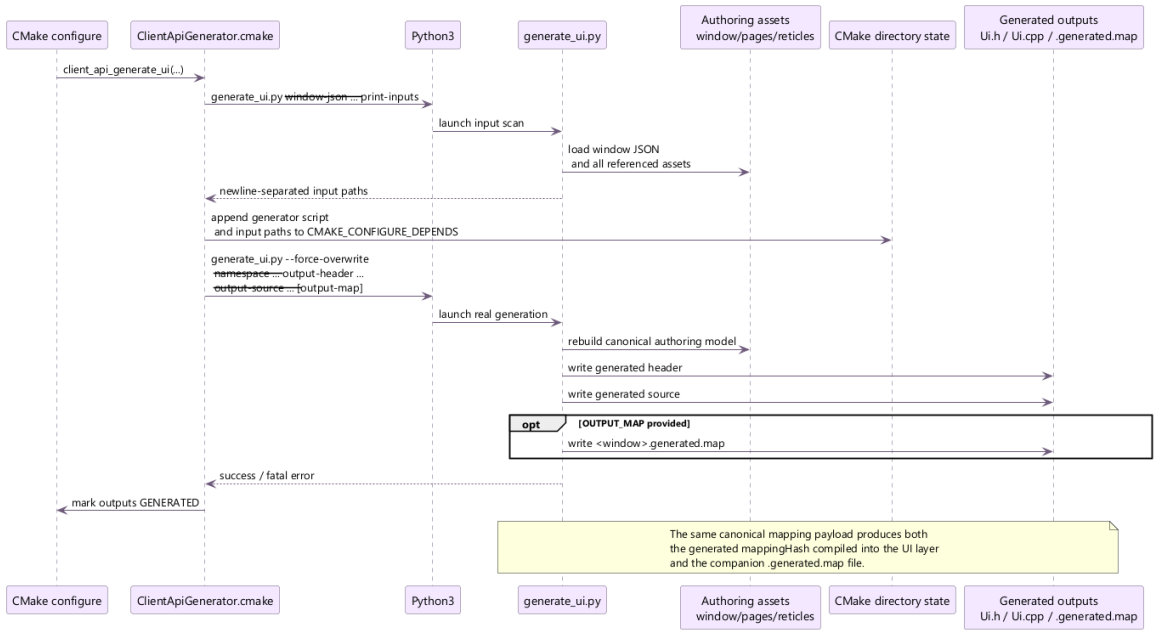
■ 5.3 client_api_generate_ui arguments

Argument	Mandatory	Integration meaning
WINDOW_JSON	yes	authored root window to parse
OUTPUT_HEADER	yes	generated typed API header

OUTPUT_SOURCE	yes	generated typed API source
OUTPUT_MAP	recommended	generated transport sidecar used by runtime and raw helpers
NAMESPACE	no	namespace exposed to the client target
UI_CLASS_NAME	no	explicit UI root class name
PAGE_CLASS_SUFFIX	no	suffix appended to generated page wrapper names
UI_CLASS_SUFFIX	no	suffix appended to the UI root if UI_CLASS_NAME is omitted
HEADER_INCLUDE	no	include path written into the generated .cpp
CONFIGURE_DEPENDS	no	extra paths that must also trigger CMake reconfigure

The important point is that `WINDOW_JSON` is only the entry point. The real dependency set is broader, which is why the input-scan mode exists.

5.4 What the CMake macro really does



Client API generation sequence

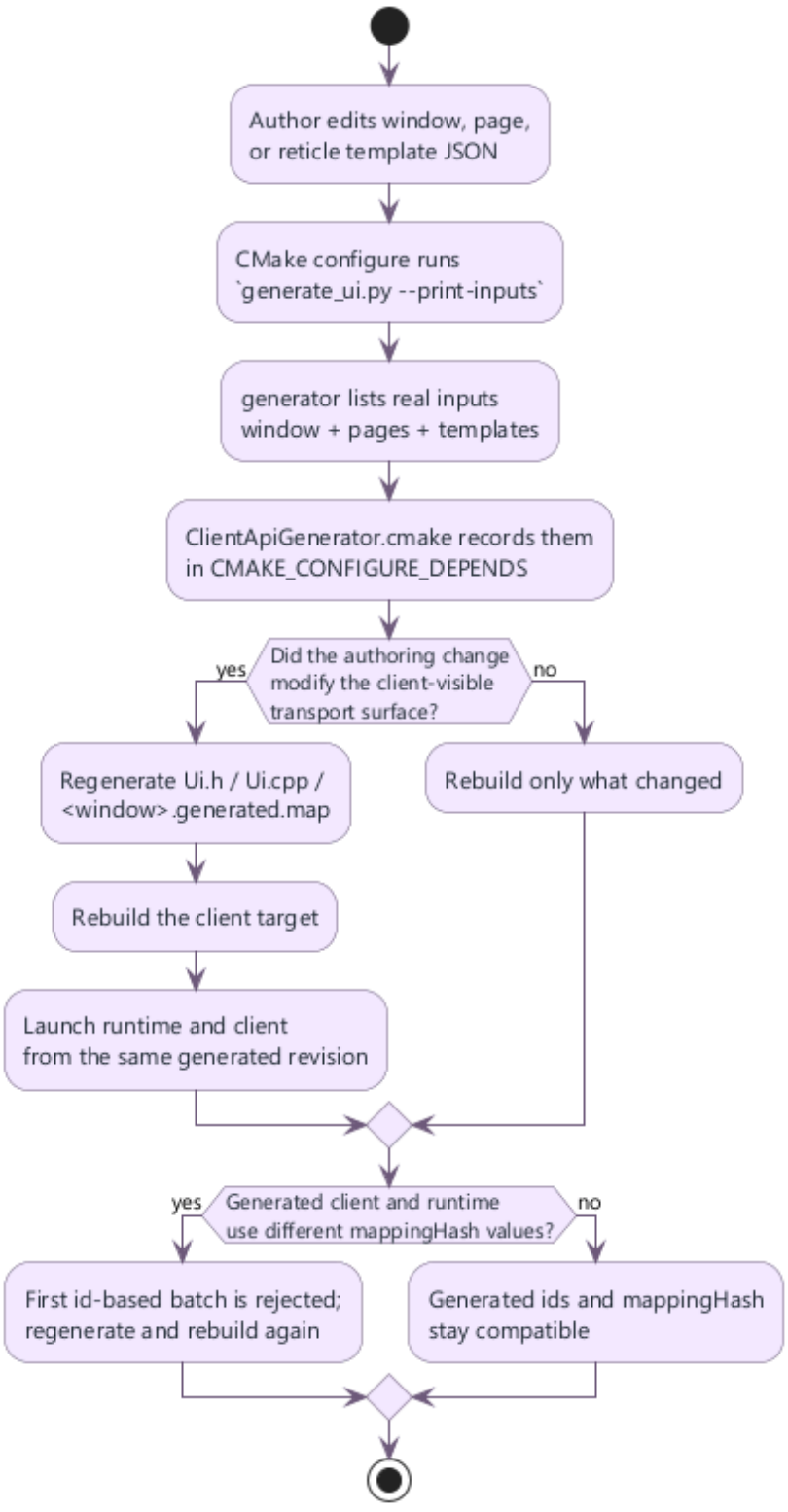
The macro performs two distinct passes:

26. 1. a configure-time scan through `generate_ui.py --print-inputs`
27. 2. a real generation pass through `generate_ui.py --force-overwrite ...`

The input scan discovers the actual dependency graph behind the window:

- the root window JSON
- every referenced page JSON
- every template file in the active reticle library

Those paths are appended to `CMAKE_CONFIGURE_DEPENDS`, which means CMake knows that an authoring change can invalidate the generated client contract even if no C++ file changed.



Developer workflow from authoring change to regenerated client API

5.5 Why --print-inputs matters for incremental builds

The generator supports a discovery mode:

CODE POWERSHELL

```
py -3 mfd_client_api/generator/scripts/generate_ui.py `
  --window-json assets/windows/demo_pages_cockpit.json `
  --output-header generated/MockupUi.h `
  --output-source generated/MockupUi.cpp `
  --print-inputs
```

The output is a newline-separated list of absolute input paths. `ClientApiGenerator.cmake` captures that stdout and registers each path in `CMAKE_CONFIGURE_DEPENDS`.

Why this matters in practice:

- if a page file changes, the generated API is invalidated
- if a reticle template changes, the generated API is invalidated
- if a new dynamic template binding is added to a page, the generated API is invalidated

Without `--print-inputs`, CMake would only see the top-level generator invocation. It would not know that editing a page or template changed the transport surface behind the generated code.

⬡ API	`--print-inputs` is part of the build contract, not a debugging option. Removing it from the CMake path makes stale generated code much more likely.

5.6 What breaks if you do not regenerate

The most visible failure mode is a `mappingHash` mismatch after an authored change that altered the transport surface:

- page rename
- static reticle rename
- exposed primitive rename
- dynamic template binding added or removed
- blink type rename

Typical sequence:

28. 1. an author changes the authored model
29. 2. `mfd_window` loads the new window JSON and the new `.generated.map`
30. 3. the client executable is still compiled against the old generated `Ui.cpp`
31. 4. the first id-based batch is rejected before the runtime mutates any scene state

Concrete example:

CODE DIFF

```
// Scenario: the client still ships yesterday's generated UI after a page rename.
- batch.mappingHash = "3852bb1a1250284ed4db4ed38d08ea5da4a2044632441bc7afdac7d5cf5885e4";
+ batch.mappingHash = "91a4d9cd12cb6b75c96c0f8f1a3d8d9f6dd8c0f6d2d9bb4a60ee8d8f4261d4ab";

- runtime error: "Generated transport map hash mismatch between the client batch and the
runtime window"
+ after regeneration: batch accepted and generated ids resolve again
```

Two related mismatch cases matter operationally:

Failure point	Exact symptom	What it usually means
client-side normalization	Command batch mappingHash does not match the generated transport map configured on the client	the executable loads a newer .generated.map but still embeds older generated C++
runtime command processor	Generated transport map hash mismatch between the client batch and the runtime window	the client batch was built from a different generated revision than the runtime window

If the transport surface changed, the only correct fix is:

32. 1. regenerate `Ui.h`, `Ui.cpp`, and `.generated.map`
33. 2. rebuild the client target
34. 3. relaunch the runtime with the same authored asset revision

■ 5.7 Rules for exposing primitives

The generated API should model operational control, not authoring noise.

Good exposure candidates:

- labels updated from live avionics data such as heading, Mach, or threat text
- geometry that the client must steer at runtime such as a command bug, a cue line, or a steering symbol
- dynamic template families that represent tracks, threats, waypoints, or cues

Bad exposure candidates:

- decorative lines that never move independently
- duplicated framing labels
- unstable implementation-only primitives that have no meaning to the integrator

Practical rule: if the client code would never mention the primitive in a requirements review, do not expose it.

■ 5.8 What the generated API contains

The generated output builds one typed object tree per authored window:

- one UI root class such as `CockpitMockupUi`
- one page wrapper per page
- one static reticle wrapper per static reticle
- one specialized handle per exposed primitive
- one generated dynamic-reticle set per page/template binding
- one optional `strobe` handle per authored page strobe
- one feedback layer that drives `IsActive()` and `IsStrobeCaptured()`

This is why the generated path should be the default integration path. The business code manipulates typed handles and leaves transport ids, runtime dynamic ids, and patch serialization to the generated layer.

5.9 What the .generated.map sidecar contains

The `.generated.map` file is not redundant output. It is the canonical transport description used by both the client and the runtime.

Table	Runtime meaning
pages	page transport ids and default page marker
reticles	static reticle ids keyed by page
primitives	exposed primitive ids keyed by reticle or template
templates	generated ids for dynamic templates
blinkTypes	generated ids for per-page blink types
mappingHash	canonical compatibility fingerprint for the whole transport surface

Keep the sidecar next to the authored window. When the client also needs raw-name helpers, the same map lets `CommandClient` normalize names and detect stale compiled code.

5.10 Example of integration into a CMake target

CODE CMAKE

```
client_api_generate_ui(
    WINDOW_JSON "assets/windows/demo_pages_cockpit.json"
    OUTPUT_HEADER "${CMAKE_CURRENT_SOURCE_DIR}/generated/MockupUi.h"
    OUTPUT_SOURCE "${CMAKE_CURRENT_SOURCE_DIR}/generated/MockupUi.cpp"
    OUTPUT_MAP "${MFD_ROOT_DIR}/assets/windows/demo_pages_cockpit.generated.map"
    NAMESPACE "mockup_ui"
    UI_CLASS_NAME "CockpitMockupUi"
    HEADER_INCLUDE "MockupUi.h")

add_executable(client_mission
    src/main.cpp
    generated/MockupUi.cpp)

target_include_directories(client_mission
    PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}/generated)

target_link_libraries(client_mission
    PRIVATE
        mfd_client_api
        mfd_common_api
        mfd_api)
```

The generated `.cpp` belongs in the target sources. Do not treat the generated layer as header-only: the source file carries the baked `mappingHash`, generated ids, feedback glue, and batch builders.

5.11 When to regenerate

Regenerate whenever the authored transport surface changes.

Must regenerate:

- page names
- static reticle ids
- exposed primitive ids
- template ids used by dynamic bindings
- dynamic page bindings
- page blink types
- the default page when startup behavior depends on it

Usually does not require regeneration:

- purely decorative geometry changes that do not affect exposed ids
- static color or layout changes on content that is not exposed to the client
- runtime logic changes inside the client executable only

Safe rule: if the client-visible contract changed, regenerate before the next compile.

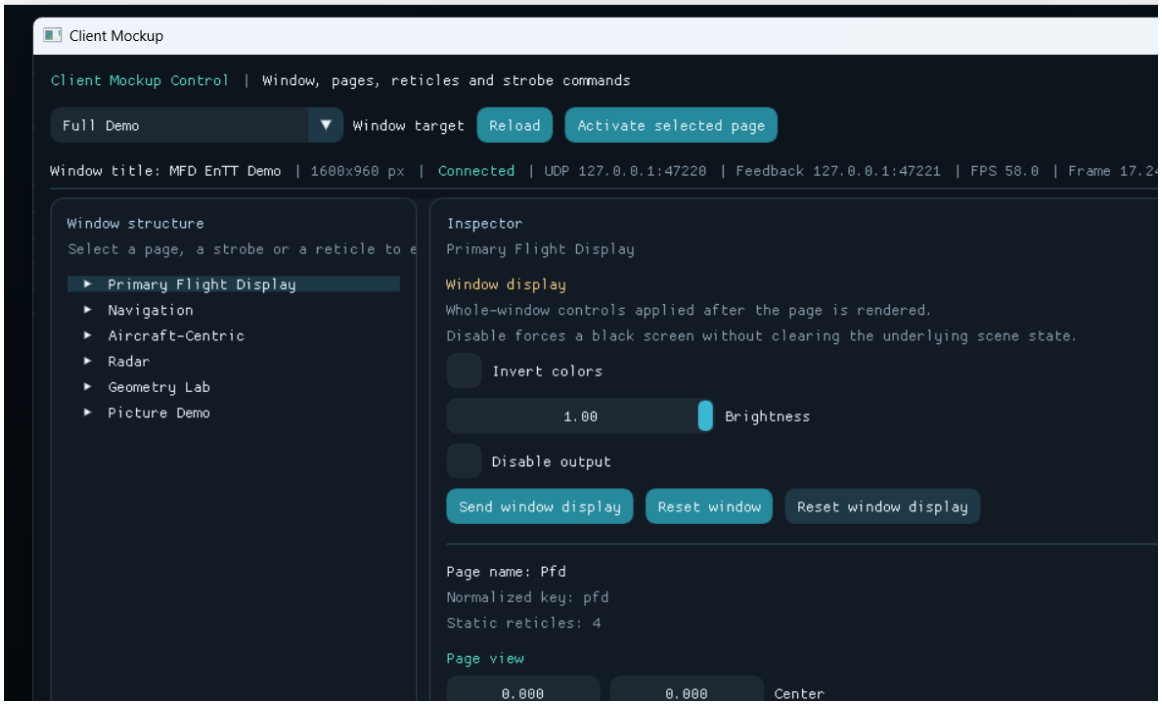
5.12 Troubleshooting the generator

Symptom	Likely cause	Fix
generated header does not expose a new accessor	the primitive was not exposed or the build reused stale outputs	expose the primitive if needed, then rerun generation
runtime rejects the first batch with a hash mismatch	generated C++ and runtime assets are from different revisions	regenerate outputs, rebuild the client, relaunch with matching assets
CMake did not rerun generation after a template edit	the build bypassed client_api_generate_ui(...) or --print-inputs was removed	restore the official macro flow
generator fails with duplicate generated names	two authored ids normalize to the same C++ name	rename the authored ids to distinct stable names
generator fails on unknown dynamic layer	dynamicReticleBindings[*].layerId references a layer not declared in the page	declare the layer or fix the binding
generator refuses to overwrite outputs	the Python entry point was run directly without --force-overwrite	use the CMake macro or pass --force-overwrite intentionally

⚠ WARNING

The generator is not cosmetic. It defines the transport contract between authored assets, runtime, and client code. Any authoring change that affects that contract must be treated like an interface change.

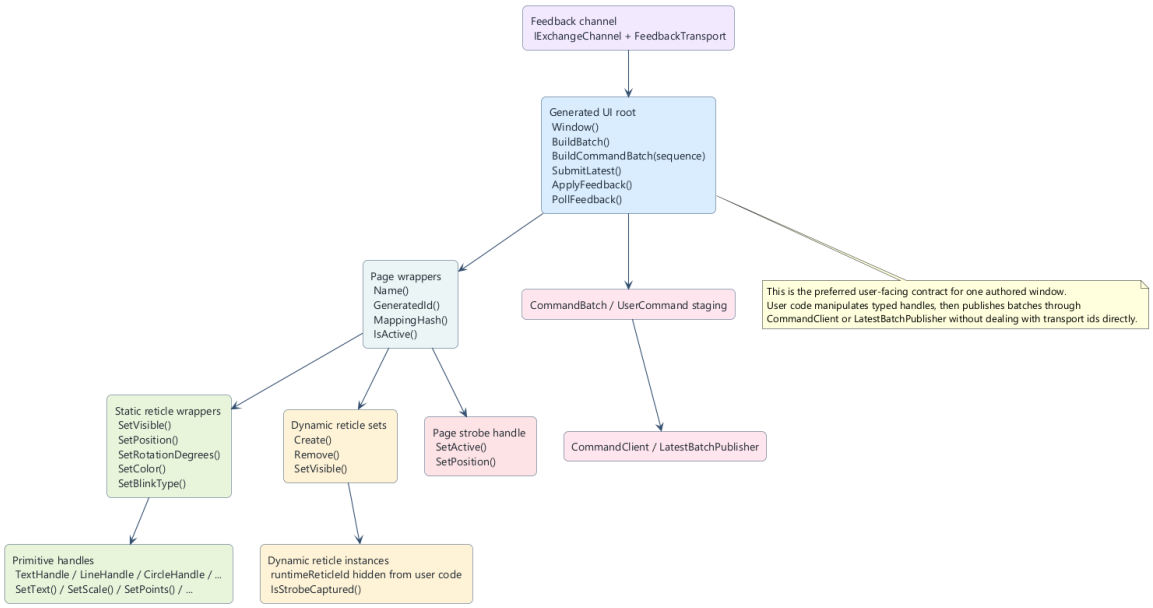
6. Use the generated API to communicate with a window



`client_mockup` runtime client screenshot

This chapter focuses on the recommended client pattern for a real integration: local state mutation first, coherent batch publication second, feedback-driven gating third.

6.1 Two layers to keep separate



Generated API user contract

Keep these two layers conceptually separate:

- the generated typed API, which models one authored window as pages, reticles, primitive handles, dynamic sets, and feedback helpers

- the transport layer `mfd::CommandClient`, which serializes and sends `UserCommand` or `CommandBatch` payloads

The intended use is:

35. 1. mutate generated handles locally
36. 2. let the generated layer collect only dirty state
37. 3. build one `CommandBatch`
38. 4. publish it through `CommandClient` or `LatestBatchPublisher`

⚠ WARNING	Wrong: call `client.Send(...)` or `client.SendBatch(...)` after every `SetText()` and `SetPosition()`. Right: update the full frame locally, then send one coherent batch for that frame.

■ 6.2 Load the authored window, transport, and feedback channel

Start from the real authored window file emitted by the project:

CODE CPP

```cpp
// Scenario: bootstrap a cockpit client that will drive HUD and radar content.
#include "MockupUi.h"
#include "mfd/control/FeedbackTransport.h"
#include "mfd/io/JsonLoader.h"

mfd::JsonLoader loader;
const mfd::LoadedWindowConfiguration loaded =
    loader.LoadWindowConfiguration(std::string(mockup_ui::CockpitMockupUi::WindowFile()));

if (!loaded.window.commandTransports.udp.has_value())
{
    throw std::runtime_error("The window does not expose an enabled UDP command transport");
}
if (!loaded.generatedTransportMap.has_value())
{
    throw std::runtime_error("The window does not expose a generated transport map");
}

std::unique_ptr<mfd::IExchangeChannel> feedbackChannel;
if (loaded.window.feedbackTransports.udp.has_value())
{
    feedbackChannel =
mfd::CreateFeedbackReceiverChannel(*loaded.window.feedbackTransports.udp);
}
```

If the generated map is missing, the generated path loses one of its main safety rails. You can still compile code that includes the generated header, but you can no longer prove transport compatibility through the map sidecar.

■ 6.3 Create CommandClient from the generated transport map

CODE CPP

```cpp
// Scenario: create the transport bridge used by the cockpit control loop.
mfd::CommandClient client(
    *loaded.window.commandTransports.udp,
    loaded.generatedTransportMap);
```

```
if (!client.IsReady())
{
    throw std::runtime_error("Unable to initialize the command client: " +
client.LastError());
}
```

`CommandClient` owns:

- transport readiness
- serialization
- UDP payload splitting when one batch exceeds the configured packet size
- normalization of generated ids and named authored ids
- compatibility checks against the loaded transport map

6.4 UI root, startup, and authored default view

The generated UI root centralizes the typed surface for one authored window:

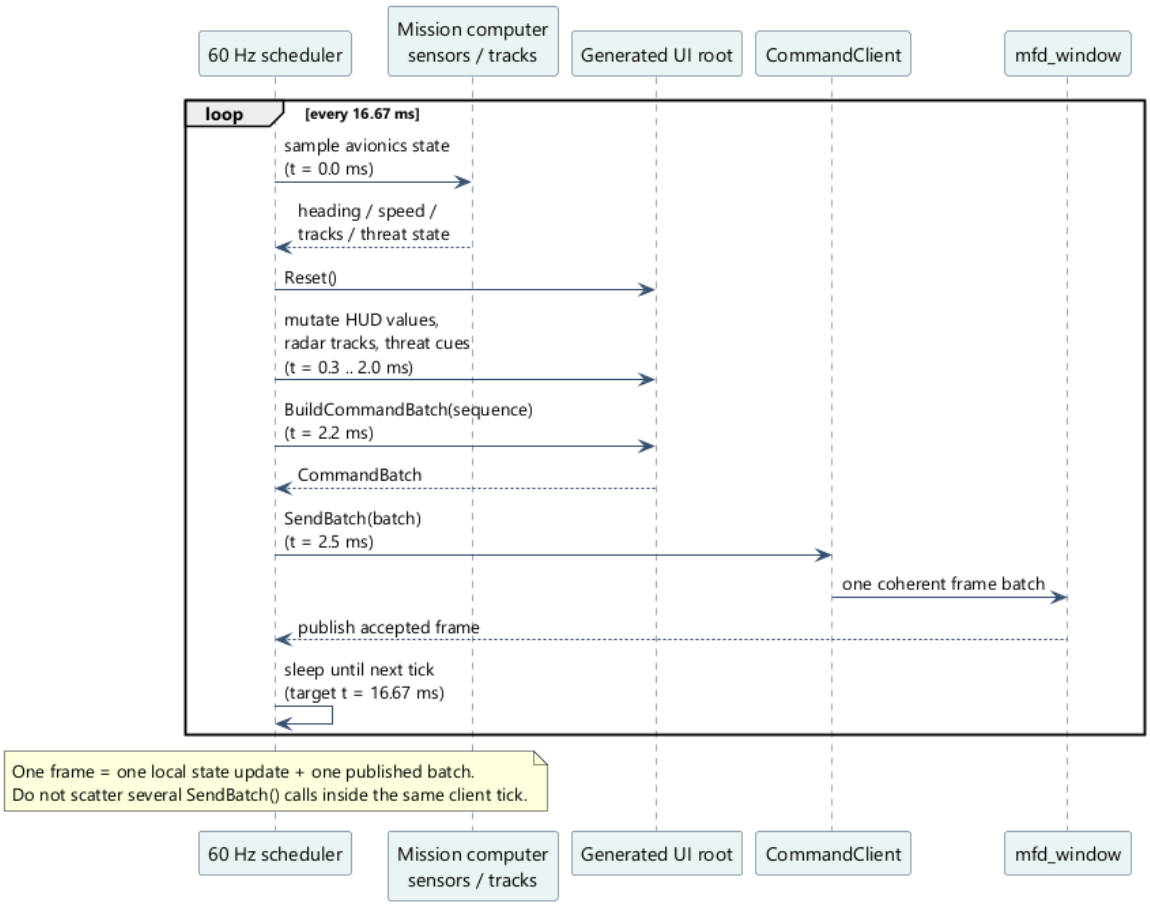
CODE CPP

```
// Scenario: prime the cockpit window before entering the realtime loop.
mockup_ui::CockpitMockupUi ui;
auto& cockpit = ui.Cockpit();

if (!ui.SendStartup(
        client,
        mfd::PageViewState {{0.0f, 0.0f}, 1.0f},
        std::string {"WP-03 READY | awaiting first radar frame"}))
{
    throw std::runtime_error("Unable to send startup batch: " + client.LastError());
}
```

`SendStartup(...)` is useful when your integration needs one predictable initial page, one predictable authored view, and one initial status caption before the continuous loop starts.

⚠ WARNING	Wrong: send startup page activation, page view, and initial labels through three unrelated command bursts. Right: use one explicit startup sequence so the runtime transitions from cold start to the first meaningful frame atomically.

6.5 Real-time loop pattern at 60 Hz



60 Hz generated client loop with one batch per frame

For a classic external avionics client, the correct pattern is one loop at 60 Hz:

39. 1. sample external state
40. 2. mutate generated handles
41. 3. call `BuildCommandBatch(sequence)`
42. 4. publish the batch once
43. 5. sleep until the next tick

CODE CPP

```cpp
// Scenario: 60 Hz cockpit loop publishing HUD values, radar contacts, and status text.
using clock = std::chrono::steady_clock;
constexpr auto kTick = std::chrono::milliseconds(16);

std::uint32_t sequence = 1U;
auto nextTick = clock::now();

while (running)
{
    const auto frameStart = clock::now();                              // t = 0.0 ms
    const MissionSample sample = ReadMissionComputer();                // t = 0.2 ms

    ui.Reset();                                                        // t = 0.3 ms
    PopulateHud(ui.Cockpit(), sample.hud);                             // t = 0.9 ms
    PopulateRadar(ui.Cockpit(), sample.radarTracks);                   // t = 1.8 ms
    ui.Cockpit().SetStatusCaption("WP-03 PUSH | THREAT SA-10");        // t = 2.0 ms

    const mfd::CommandBatch batch = ui.BuildCommandBatch(sequence); // t = 2.2 ms
    if (!client.SendBatch(batch))                                      // t = 2.5 ms
```

```
    {
        throw std::runtime_error("Unable to send frame batch: " + client.LastError());
    }

    ++sequence;
    nextTick += kTick;

    if (const auto now = clock::now(); now < nextTick)
    {
        std::this_thread::sleep_until(nextTick);                    // t = 16.6 ms target
    }
    else
    {
        nextTick = now;
    }
}
```

This is the reference pattern because the runtime processes a coherent per-frame command set. Scattering multiple send calls through the frame destroys sequencing, increases packet count, and makes feedback interpretation harder.

⚠ WARNING	Wrong: one `SendBatch()` for HUD text, another for page view, another for radar tracks. Right: one batch per client frame, one `sequence`, one coherent state transition.

■ 6.6 Page wrappers and authoritative active-page feedback

Each generated page wrapper exposes authored identity and authoritative active-page feedback:

- `Name()`
- `GeneratedId()`
- `MappingHash()`
- `IsActive()`

`IsActive()` is intentionally feedback-driven. It becomes true only when the runtime reports that the page is currently rendered as active. It does not become true merely because your client requested page activation.

This distinction matters for gating client logic. If your application should only publish page-specific overlays when the page is actually being rendered, gate that behavior on `IsActive()` rather than on your own last requested page variable.

■ 6.7 Window-level display controls

The UI root exposes `Window()` for whole-window state that is not tied to one page:

CODE CPP

```
// Scenario: dim the entire window for a night attack profile.
ui.Window().SetColorInverted(false);
ui.Window().SetBrightness(0.42f);
ui.Window().SetDisabled(false);

if (!client.SendBatch(ui.BuildCommandBatch(104U)))
{
```

```
    throw std::runtime_error(client.LastError());
}
```

Use this layer for whole-window presentation state such as brightness, inversion, or blackout. Do not overload reticle-level color changes when the real operational intent is a window-wide display mode change.

■ 6.8 Static reticle wrappers for persistent avionics values

Static reticle wrappers are the right surface for authored elements that always exist on the page and only need live value updates.

CODE CPP

```
// Scenario: update persistent HUD boxes with real mission values.
auto& cockpit = ui.Cockpit();

cockpit.hudSpeedBox.SetValue("420");
cockpit.hudMachBox.SetValue("0.64");
cockpit.hudHeadingBox.SetValue("045");
cockpit.hudFpaBox.SetValue("+02.5");
cockpit.hudThrottleBox.SetValue("88%");
cockpit.hudRadarBox.SetValue("EMIT");

if (overspeed)
{
    cockpit.hudSpeedBox.Blink = cockpit.overspeed;
    cockpit.hudMachBox.Blink = cockpit.overspeed;
}
else
{
    cockpit.hudSpeedBox.Blink = nullptr;
    cockpit.hudMachBox.Blink = nullptr;
}
```

This pattern keeps authored structure static while letting the client update operational values every frame.

⚠ WARNING	Wrong: rebuild the authored HUD layout as dynamic reticles just to update speed, heading, or throttle text. Right: keep persistent authored elements static and use the generated static wrappers as the live control surface.

Common reticle surface

Method	Use
SetVisible(bool)	show or hide the reticle
SetBlinkEnabled(bool)	toggle blinking without changing the blink type
SetBlinkType(const BlinkType&)	use one explicit page blink type
ClearBlinkType()	fall back to the page default blink behavior

SetPosition(Vec2)	move the whole reticle
SetRotationDegrees(float)	rotate the whole reticle
SetColor(ColorRgba)	override stroke color
SetThickness(float)	override line thickness
SetText(std::string)	update the first text primitive or generated status primitive
SetLetterSpacing(float)	update authored text spacing

■ 6.9 Primitive handles for exposed geometry and text

Primitive handles are what make the generated API feel like an integration API instead of a string-based patch list. They let the client touch only the authored surface that was explicitly meant to move.

CODE CPP

```
// Scenario: steer the heading command bug and bank pointer from flight guidance.
auto& cockpit = ui.Cockpit();

cockpit.adiHeadingCommandBug.CommandBug().SetRotationDegrees(+27.0f);
cockpit.adiBankPointer.Pointer().SetRotationDegrees(-12.0f);
cockpit.adiHeadingBox.CommandValue().SetText("072");
cockpit.adiHeadingBox.HeadingValue().SetText("045");
```

Use primitive handles when the client needs one sub-element to move independently from the reticle that owns it. That is typical for command bugs, cue lines, text fields, and dynamic geometry markers.

Common primitive surface

Method	Use
SetVisible(bool)	toggle only this primitive
SetPosition(Vec2)	local translation inside the reticle
SetRotationDegrees(float)	local rotation
SetScale(Vec2)	local scale
SetColor(ColorRgba)	stroke color
SetFillColor(ColorRgba)	fill color
SetFilled(bool)	fill toggle
SetThickness(float)	line thickness
SetLineStyle(LineStyle)	line style override

Specialized surfaces

Handle	Extra methods
TextHandle	SetText, SetLetterSpacing
TimeHandle	SetLetterSpacing
LineHandle	SetStart, SetEnd
CircleHandle	SetRadius
RingHandle	SetInnerRadius, SetOuterRadius, SetSegments
RectangleHandle	SetWidth, SetHeight, SetSize
EllipseHandle	SetWidth, SetHeight, SetSize
SquareHandle	SetWidth, SetHeight, SetSize
DiamondHandle	SetWidth, SetHeight, SetSize
TriangleHandle	SetPoints
PolylineHandle	SetPoints, SetClosed
BezierHandle	SetControlPoints, SetSegments
ArcHandle	SetRadius, SetStartAngleDegrees, SetEndAngleDegrees, SetSegments
ImageHandle	common primitive surface only
⚠ WARNING	Wrong: expose every decorative primitive and then patch them from business code. Right: expose only the sub-elements that correspond to a real runtime control requirement.

■ 6.10 Dynamic reticles for radar tracks, threats, and steering cues

Dynamic reticles are the right model for runtime entities that appear, move, and disappear independently from the static page layout.

CODE CPP

```cpp
// Scenario: publish two live radar contacts and one threat cue on the cockpit page.
auto& cockpit = ui.Cockpit();
auto& contacts = cockpit.DynamicCockpitRadarContact();

auto& lead = contacts.Create();
lead.SetVisible(true);
lead.SetPosition({1.18f, 0.17f});
```

```
lead.SetRotationDegrees(-18.0f);
lead.SetColor({86, 244, 162, 255});
lead.ContactLabel().SetText("BRAA 045/32");

auto& threat = contacts.Create();
threat.SetVisible(true);
threat.SetPosition({0.92f, -0.08f});
threat.SetRotationDegrees(+36.0f);
threat.SetColor({255, 198, 109, 255});
threat.ContactLabel().SetText("THREAT SA-10");
threat.Blink = cockpit.threat;

cockpit.radarStatusBox.SetValue("THREAT SA-10");
cockpit.SetStatusCaption("WP-03 PUSH | COMMIT NORTH");
```

To remove one dynamic instance, remove the handle from the generated set and let the next batch carry the removal command:

CODE CPP

```
// Scenario: delete a stale contact when the track drops from the tactical picture.
contacts.Remove(threat);
```

The generated set hides three pieces of bookkeeping that user code should not own:

- the runtime reticle identifier
- the template transport id
- the `mappingHash` carried by the final batch

⚠ WARNING	Wrong: invent your own runtime-reticle-id scheme outside the generated set. Right: let `Create()` and `Remove()` manage lifecycle commands, then publish the resulting batch once per frame.

■ 6.11 Build one coherent batch per frame

The two generated batch builders exist for different levels of transport control:

- `BuildBatch()` returns raw `std::vector<mfd::UserCommand>`
- `BuildCommandBatch(sequence)` returns one `mfd::CommandBatch` with both `sequence` and `mappingHash`

For production code, prefer the second form:

CODE CPP

```
// Scenario: publish one frame-aligned command batch with sequence tracking.
const mfd::CommandBatch batch = ui.BuildCommandBatch(sequence);
if (!client.SendBatch(batch))
{
    throw std::runtime_error(client.LastError());
}
```

Use `sequence` when the client operates in explicit update cycles. The runtime uses `sequence` together with `mappingHash` to reject stale or duplicate batches.

■ 6.12 When to prefer LatestBatchPublisher

Use `LatestBatchPublisher` when the client continuously computes the latest state and only the freshest unsent frame matters.

Typical fit:

- radar sweeps
- HUD state streams
- synthetic moving maps
- continuously refreshed flight-symbology clients

Poor fit:

- low-rate command tools where every transaction must be delivered
- one-shot administrative commands
- flows where dropping intermediate pending frames is unacceptable

CODE CPP

```cpp
// Scenario: stream the freshest cockpit frame without blocking the producer loop.
mfd::client::LatestBatchPublisher publisher(*loaded.window.commandTransports.udp);
if (!publisher.IsReady())
{
    throw std::runtime_error("Unable to create realtime publisher: " +
publisher.LastError());
}

if (!ui.SubmitLatest(publisher, sequence))
{
    throw std::runtime_error("Unable to queue latest frame: " + publisher.LastError());
}
```

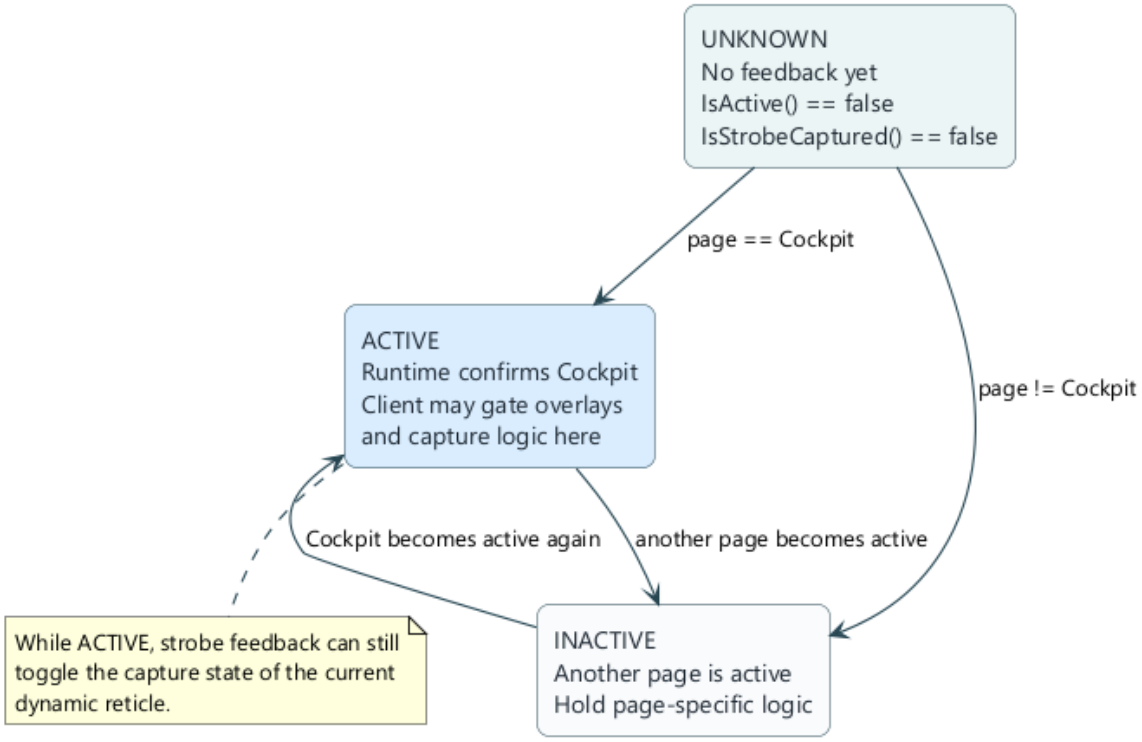
Implementation-backed behavior to know:

- one dedicated worker thread serializes sends
- `SubmitLatest()` replaces any older pending unsent batch with the newest one
- dynamic reticle lifecycle commands are preserved across pending-batch replacement when the `mappingHash` stays the same
- `Flush()` blocks until the current send completes and no pending batch remains
- `Stop()` drops any unsent pending batch and terminates the worker thread

This makes `LatestBatchPublisher` a state-stream helper, not a guaranteed delivery queue.

⚠ WARNING	Wrong: use `LatestBatchPublisher` for every command path in the application. Right: reserve it for high-rate "latest state wins" publishing, and keep plain `SendBatch()` for explicit control transactions.

6.13 Feedback-driven state machine



Feedback-driven page activity and capture state machine

`IsActive()` and `IsStrobeCaptured()` should gate client behavior, not just feed telemetry logs.

CODE CPP

```cpp
// Scenario: only commit the intercept logic when the runtime confirms both page activity
and strobe capture.
std::string feedbackError;
if (feedbackChannel)
{
    ui.PollFeedback(*feedbackChannel, 8U, &feedbackError);
}

auto& cockpit = ui.Cockpit();

if (!cockpit.IsActive())
{
    HoldRadarOverlay();
    return;
}

if (track != nullptr && track->IsStrobeCaptured())
{
    AuthorizeCommit("THREAT SA-10");
    cockpit.SetStatusCaption("WP-03 PUSH | CAPTURE CONFIRMED");
}
else
{
    ContinueSearch();
    cockpit.SetStatusCaption("WP-03 PUSH | SEARCHING");
}
```

Both signals are authoritative runtime feedback:

- `IsActive()` means the runtime is actually rendering that page

- `IsStrobeCaptured()` means the runtime confirmed that exact dynamic instance as the captured target

⚠ WARNING	Wrong: assume capture as soon as the client places the strobe on top of a track. Right: wait for the runtime feedback path to confirm the active page and the captured runtime reticle.

■ 6.14 What feedback guarantees

The generated feedback helpers intentionally expose only authoritative runtime state:

- `Page::IsActive()` is false until `ActivePageFeedback` is received
- `DynamicReticle::IsStrobeCaptured()` is false until the strobe feedback identifies that exact runtime reticle
- stale feedback sequences are ignored by the runtime-feedback tracker
- if the feedback channel is disabled, these helpers remain conservative rather than speculative

This makes the generated feedback surface safe to use in client state machines.

■ 6.15 Client sanitization and hygiene

The generated layer makes publication easier, but it does not sanitize domain data for you. A robust client still needs to validate its own upstream inputs.

Recommended safeguards:

- clamp logical coordinates to the authored range you accept
- reject or normalize non-finite floats before touching generated handles
- clamp brightness to `[0, 1]`
- cap loop delta time to avoid huge catch-up steps after a debugger stop
- keep one clear owner for dynamic-reticle lifecycle decisions
- surface `client.LastError()` and `publisher.LastError()` in operational logs

If your upstream data can contain `NaN`, `Inf`, or impossible kinematic values, sanitize before mutating the generated state tree.

■ 6.16 When the raw API is still appropriate

The generated layer should be the default for window-specific integrations, but the raw `CommandClient` helpers still have a place:

- generic tooling that targets several windows
- low-level debugging tools
- migration code that predates the generated client surface

CODE CPP

```cpp
// Scenario: a generic tactical tool injects one named dynamic reticle without including the
generated header.
mfd::ReticlePatch patch;
patch.visible = true;
patch.position = mfd::Vec2 {0.24f, -0.11f};
patch.text = std::string {"BRAA 045/32"};
```

```
if (!client.UpsertDynamicReticle("Cockpit", "lead_track", "cockpit_radar_contact", patch))
{
    throw std::runtime_error(client.LastError());
}
```

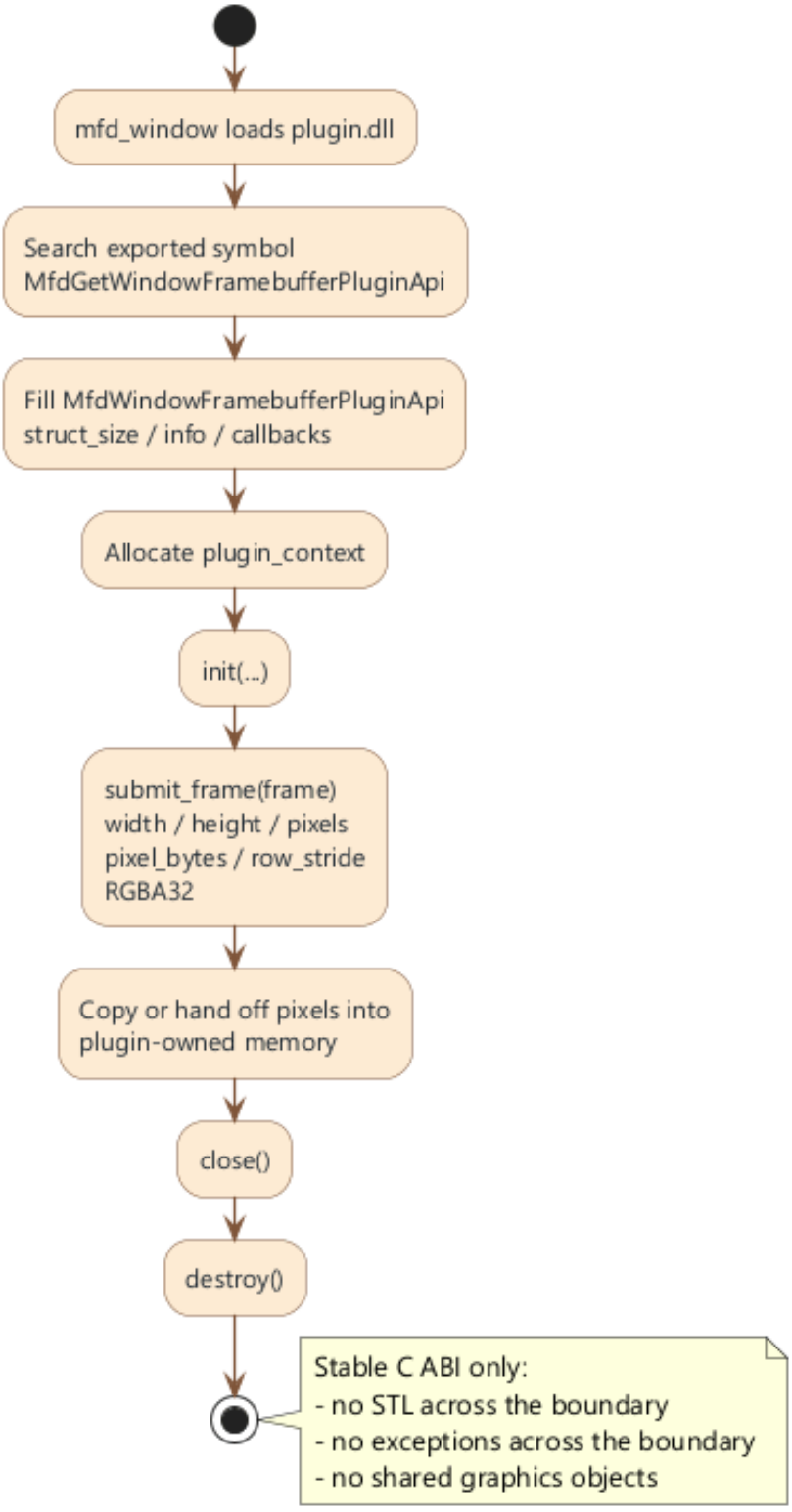
Use the raw path for genericity, not as a substitute for the generated path in a window-specific product client.

6.17 Minimal end-to-end example

CODE CPP

```
// Scenario: minimum typed client that activates the cockpit page and publishes one radar-
ready frame.
#include "MockupUi.h"
#include "mfd/control/CommandClient.h"
#include "mfd/io/JsonLoader.h"

int main()
{
    mfd::JsonLoader loader;
    const mfd::LoadedWindowConfiguration loaded =

loader.LoadWindowConfiguration(std::string(mockup_ui::CockpitMockupUi::WindowFile()));

    if (!loaded.window.commandTransports.udp.has_value() ||
        !loaded.generatedTransportMap.has_value())
    {
        return 1;
    }

    mfd::CommandClient client(
        *loaded.window.commandTransports.udp,
        loaded.generatedTransportMap);
    if (!client.IsReady())
    {
        return 1;
    }

    mockup_ui::CockpitMockupUi ui;
    auto& cockpit = ui.Cockpit();

    if (!client.ActivatePage(cockpit))
    {
        return 1;
    }

    ui.Window().SetBrightness(0.65f);
    cockpit.hudHeadingBox.SetValue("045");
    cockpit.radarStatusBox.SetValue("SEARCH");
    cockpit.SetStatusCaption("WP-03 READY | BRAA 045/32");

    return client.SendBatch(ui.BuildCommandBatch(1U)) ? 0 : 1;
}
```

7. Write a framebuffer capture DLL plugin



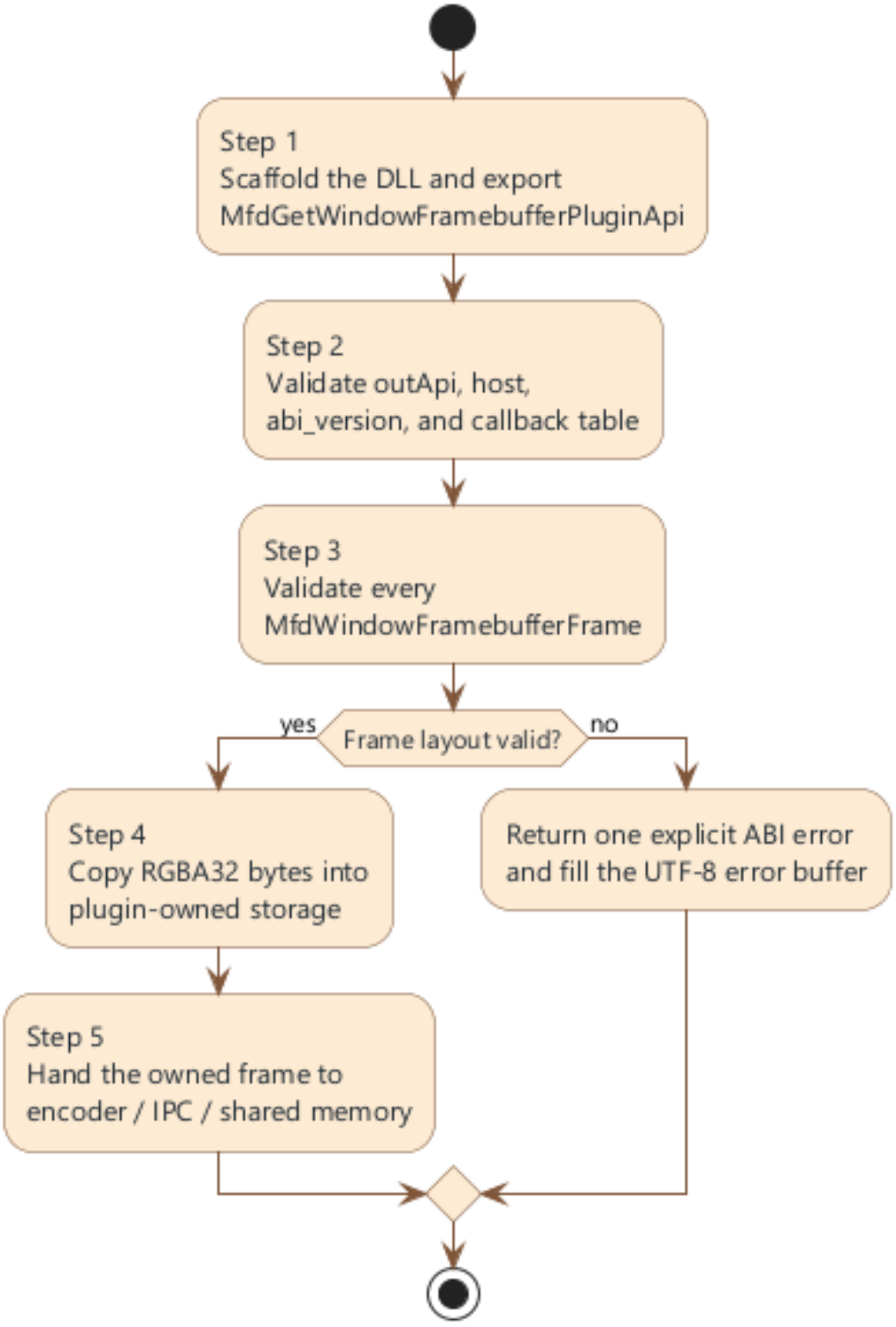
ABI framebuffer plugin

This chapter is intentionally written as a standalone integration guide. You should be able to implement, validate, and test a framebuffer plugin without reading the rest of the manual first.

Recommended order:

44. 1. scaffold the plugin and export the stable entry point

45. 2. validate the ABI contract in the factory and `init`
46. 3. validate every frame descriptor
47. 4. copy or hand off the pixels before `submit_frame` returns
48. 5. connect the async encoder or IPC path behind that handoff



Framebuffer plugin integration workflow for an application team

7.1 Why the framebuffer plugin ABI is C-only

The plugin ABI intentionally avoids C++ implementation details across the DLL boundary:

- no STL containers
- no C++ class layout assumptions
- no exceptions across the boundary
- no host-owned graphics objects
- no dependency on one compiler-specific ABI

The public header is:

```text
mfd_window_plugin_api/include/mfd/window/WindowLauncherPlugin.h
```

If you keep the DLL boundary C-only, you can freely choose your internal C++17 implementation behind that boundary.

7.2 Step 1: scaffold the plugin and export the entry point

The runtime looks for one exported symbol:

```cpp
MfdGetWindowFramebufferPluginApi
```

The public header also exposes the same name through:

```cpp
MFD_WINDOW_FRAMEBUFFER_PLUGIN_ENTRY_POINT
```

Your first deliverable is a DLL that exports that factory symbol and fills one `MfdWindowFramebufferPluginApi` structure.

7.3 Step 2: validate ABI and host contract during startup

`MfdWindowFramebufferPluginApi` is the exported callback table:

Field	Why it matters
struct_size	forward-compatibility and layout validation
info	immutable plugin metadata
plugin_context	plugin-owned opaque state
init	host-to-plugin startup callback
submit_frame	per-frame entry point
close	shutdown callback before unload
destroy	final context destruction hook

`MfdWindowFramebufferPluginInfo` describes the plugin instance:

Field	Use
struct_size	versioned metadata layout
abi_version	expected ABI revision
plugin_id	stable machine-readable plugin identifier
display_name	human-readable plugin name

At minimum, validate these conditions in the factory and `init`:

- `outApi != nullptr`
- `host != nullptr`
- `host->abi_version == MFD_WINDOW_FRAMEBUFFER_PLUGIN_ABI_VERSION`
- every mandatory callback is populated

7.4 Step 3: understand the frame descriptor you receive

`submit_frame` receives one `MfdWindowFramebufferFrame` borrowed from the host:

Field	Meaning
struct_size	versioned frame structure size
pixel_format	currently MfdWindowFramebufferPixelFormat_Rgba32
width	frame width in pixels
height	frame height in pixels
row_stride_bytes	number of bytes between two rows
pixels	borrowed pointer to the first pixel byte
pixel_bytes	total byte count available through pixels

Assume only what the ABI states. Do not infer that GPU resources are shared, that another pixel format will appear with the same layout, or that the pixel memory survives beyond the callback.

7.5 Step 4: validate every frame descriptor defensively

The ABI already exposes helpers for the most important validation steps:

Helper	Use
MfdWindowComputeFramebufferRgba32ByteCount	expected byte count for one width/height pair

MfdWindowValidateFramebufferRgba32Layout	validate raw RGBA32 layout and byte count
MfdWindowValidateFramebufferFrame	validate the full frame descriptor

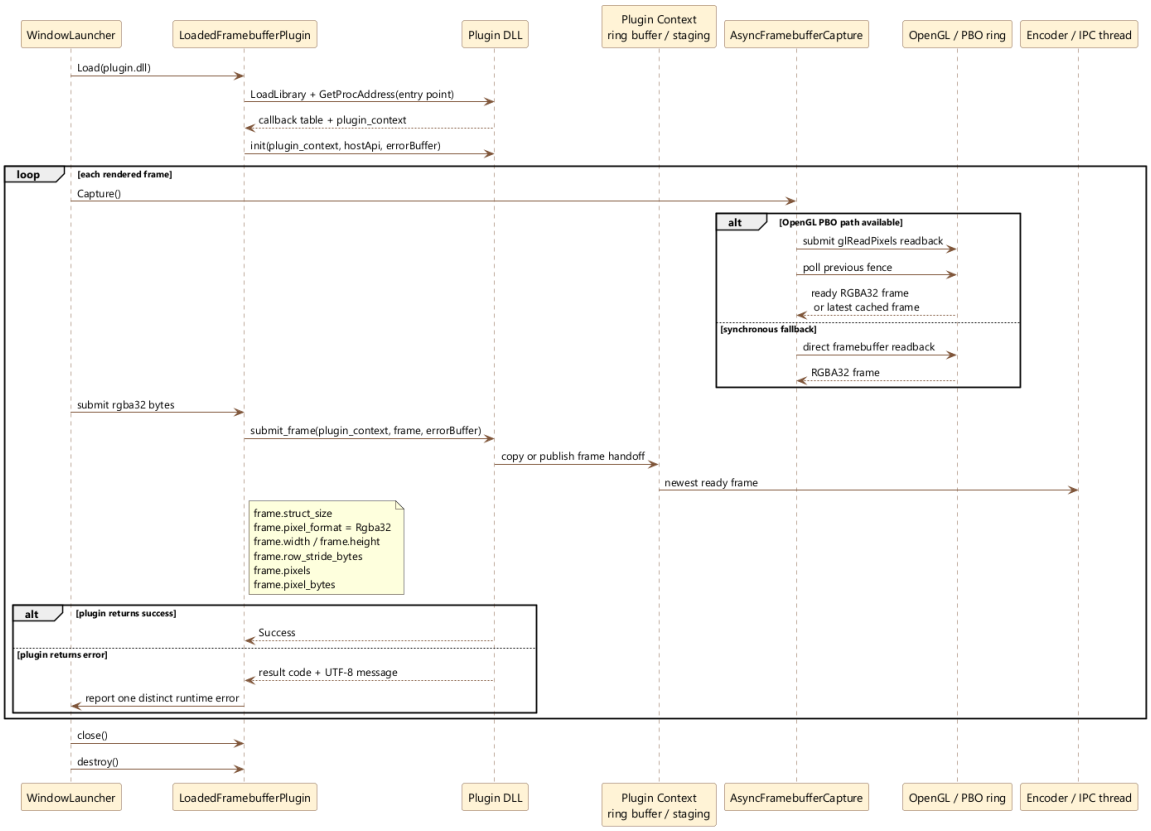
Use them at the start of `submit_frame`:

CODE CPP

```
// Scenario: reject malformed frames before touching plugin-owned buffers.
MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL SubmitFramePlugin(
    void* pluginContext,
    const MfdWindowFramebufferFrame* frame,
    MfdWindowUtf8Buffer* error) noexcept
{
    if (pluginContext == nullptr || frame == nullptr)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    if (MfdWindowValidateFramebufferFrame(frame) == 0)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    return MfdWindowFramebufferPluginResultCode_Success;
}
```

If validation fails, return one explicit ABI result code and write one human-readable error string into the provided UTF-8 buffer when possible.

7.6 Step 5: know the lifecycle before you attach real logic



Framebuffer capture runtime sequence and async frame handoff

The normal runtime lifecycle is:

49. 1. `mfd_window` loads the DLL
50. 2. the loader calls `MfdGetWindowFramebufferPluginApi`
51. 3. the plugin allocates its context and returns the callback table
52. 4. the runtime calls `init`
53. 5. the runtime calls `submit_frame` once per rendered frame
54. 6. the runtime calls `close`
55. 7. the runtime calls `destroy`

Design your context so that `init`, `submit_frame`, `close`, and `destroy` each have one clear responsibility. Do not let `submit_frame` lazily initialize half of the encoder stack unless you are forced to.

7.7 Minimal plugin scaffold

CODE CPP

```cpp
// Scenario: minimal plugin factory that validates ABI and returns one callback table.
#include "mfd/window/WindowLauncherPlugin.h"

struct PluginContext
{
    bool initialized = false;
};

MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL InitPlugin(
    void* pluginContext,
    const MfdWindowFramebufferPluginHostApi* host,
    MfdWindowUtf8Buffer* error) noexcept
{
    if (pluginContext == nullptr || host == nullptr ||
        host->abi_version != MFD_WINDOW_FRAMEBUFFER_PLUGIN_ABI_VERSION)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    static_cast<PluginContext*>(pluginContext)->initialized = true;
    return MfdWindowFramebufferPluginResultCode_Success;
}

void MFD_WINDOW_PLUGIN_CALL ClosePlugin(void* pluginContext) noexcept
{
    (void)pluginContext;
}

void MFD_WINDOW_PLUGIN_CALL DestroyPlugin(void* pluginContext) noexcept
{
    delete static_cast<PluginContext*>(pluginContext);
}

extern "C" __declspec(dllexport) MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL
MfdGetWindowFramebufferPluginApi(MfdWindowFramebufferPluginApi* outApi, MfdWindowUtf8Buffer*
error) noexcept
{
    if (outApi == nullptr)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }
```

```
    auto* context = new (std::nothrow) PluginContext();
    if (context == nullptr)
    {
        return MfdWindowFramebufferPluginResultCode_InternalFailure;
    }

    *outApi = {};
    outApi->struct_size = sizeof(*outApi);
    outApi->info.struct_size = sizeof(outApi->info);
    outApi->info.abi_version = MFD_WINDOW_FRAMEBUFFER_PLUGIN_ABI_VERSION;
    outApi->plugin_context = context;
    outApi->init = &InitPlugin;
    outApi->submit_frame = &SubmitFramePlugin;
    outApi->close = &ClosePlugin;
    outApi->destroy = &DestroyPlugin;
    return MfdWindowFramebufferPluginResultCode_Success;
}
```

This is the correct starting point: a tiny stable C boundary, and all implementation detail hidden behind `plugin_context`.

7.8 Copy pixels safely inside submit_frame

The runtime owns `frame->pixels`. Your plugin only borrows that memory during the callback.

CODE CPP

```
// Scenario: copy one RGBA32 frame into plugin-owned storage before returning.
struct PluginContext
{
    std::vector<std::uint8_t> staging;
    int width = 0;
    int height = 0;
    std::size_t stride = 0;
};

MfdWindowFramebufferPluginResultCode MFD_WINDOW_PLUGIN_CALL SubmitFramePlugin(
    void* pluginContext,
    const MfdWindowFramebufferFrame* frame,
    MfdWindowUtf8Buffer* error) noexcept
{
    if (pluginContext == nullptr || frame == nullptr ||
        MfdWindowValidateFramebufferFrame(frame) == 0)
    {
        return MfdWindowFramebufferPluginResultCode_InvalidArgument;
    }

    auto& context = *static_cast<PluginContext*>(pluginContext);
    context.staging.resize(frame->pixel_bytes);
    std::memcpy(context.staging.data(), frame->pixels, frame->pixel_bytes);
    context.width = frame->width;
    context.height = frame->height;
    context.stride = frame->row_stride_bytes;
    return MfdWindowFramebufferPluginResultCode_Success;
}
```

✕ DANGER	Never keep `frame->pixels` after `submit_frame` returns. The pointer is borrowed host memory whose lifetime is only guaranteed for the duration of the callback.

7.9 Connect an async encoder or IPC path without blocking submit_frame

Once the safe copy rule is respected, the usual next step is to hand the copied frame to another thread.

CODE CPP

```cpp
// Scenario: pseudocode for a lock-free ring buffer handoff to an encoder thread.
struct FrameSlot
{
    std::atomic<bool> ready {false};
    std::vector<std::uint8_t> pixels;
    int width = 0;
    int height = 0;
    std::size_t stride = 0;
};

struct PluginContext
{
    std::array<FrameSlot, 3> ring;
    std::atomic<std::uint32_t> writeIndex {0};
    std::atomic<std::uint32_t> readIndex {0};
};

MfdWindowFramebufferPluginResultCode SubmitFramePlugin(...) noexcept
{
    FrameSlot& slot = context.ring[context.writeIndex.load() % context.ring.size()];
    if (slot.ready.load(std::memory_order_acquire))
    {
        return MfdWindowFramebufferPluginResultCode_Success; // drop this frame instead of
blocking
    }

    slot.pixels.resize(frame->pixel_bytes);
    std::memcpy(slot.pixels.data(), frame->pixels, frame->pixel_bytes);
    slot.width = frame->width;
    slot.height = frame->height;
    slot.stride = frame->row_stride_bytes;
    slot.ready.store(true, std::memory_order_release);
    context.writeIndex.fetch_add(1, std::memory_order_relaxed);
    return MfdWindowFramebufferPluginResultCode_Success;
}

void EncoderThreadMain(PluginContext& context)
{
    for (;;)
    {
        FrameSlot& slot = context.ring[context.readIndex.load() % context.ring.size()];
        if (!slot.ready.load(std::memory_order_acquire))
        {
            continue;
        }

        EncodeOrPublish(slot.width, slot.height, slot.stride, slot.pixels);
        slot.ready.store(false, std::memory_order_release);
        context.readIndex.fetch_add(1, std::memory_order_relaxed);
    }
}
```

This pattern gives `submit_frame` a bounded job:

- validate the frame
- copy it into owned storage

- publish the ownership handoff
- return immediately

That is the right place to connect video encoding, shared memory publication, or IPC forwarding.

7.10 What not to do

Do not:

- keep `frame->pixels` beyond the callback
- throw exceptions across the ABI
- assume a pixel format other than the one validated by the helpers
- block the render loop on disk I/O, encoding, or network publication
- write into the memory exposed by `frame->pixels`

If your plugin must choose between dropping one frame and blocking the render loop, prefer the explicit drop strategy and expose it in diagnostics.

7.11 Test the plugin without the full runtime

You can unit-test most of the plugin contract without launching `mfd_window`. Build a fake frame on the stack and call your callbacks directly.

CODE CPP

```cpp
// Scenario: unit-test the plugin against one synthetic 2x2 RGBA32 frame.
std::array<std::uint8_t, 16> pixels {};

MfdWindowFramebufferFrame frame {};
frame.struct_size = sizeof(frame);
frame.pixel_format = MfdWindowFramebufferPixelFormat_Rgba32;
frame.width = 2;
frame.height = 2;
frame.row_stride_bytes = 8;
frame.pixels = pixels.data();
frame.pixel_bytes = pixels.size();

EXPECT_NE(MfdWindowValidateFramebufferFrame(&frame), 0);

PluginContext context;
MfdWindowUtf8Buffer error {};
EXPECT_EQ(
    SubmitFramePlugin(&context, &frame, &error),
    MfdWindowFramebufferPluginResultCode_Success);
EXPECT_EQ(context.staging.size(), pixels.size());
```

This isolates plugin logic from runtime launch, OpenGL capture, and DLL loading. It is the fastest way to prove:

- frame validation works
- copy logic works
- async queue handoff works
- error reporting works on malformed descriptors

7.12 Repository reference plugin

The repository already includes one minimal reference implementation:

```
examples/mfd_framebuffer_stdout_plugin/src/FramebufferStdoutPlugin.cpp
```

That sample proves:

- callback-table export
- ABI validation
- safe frame validation
- context allocation and destruction

Start from it if you need a clean skeleton, then add your own queueing, encoder integration, or IPC path.

7.13 CMake and build notes

The reference plugin is a standard C++17 DLL. The critical points are not CMake tricks; they are ABI correctness and symbol export.

Checklist:

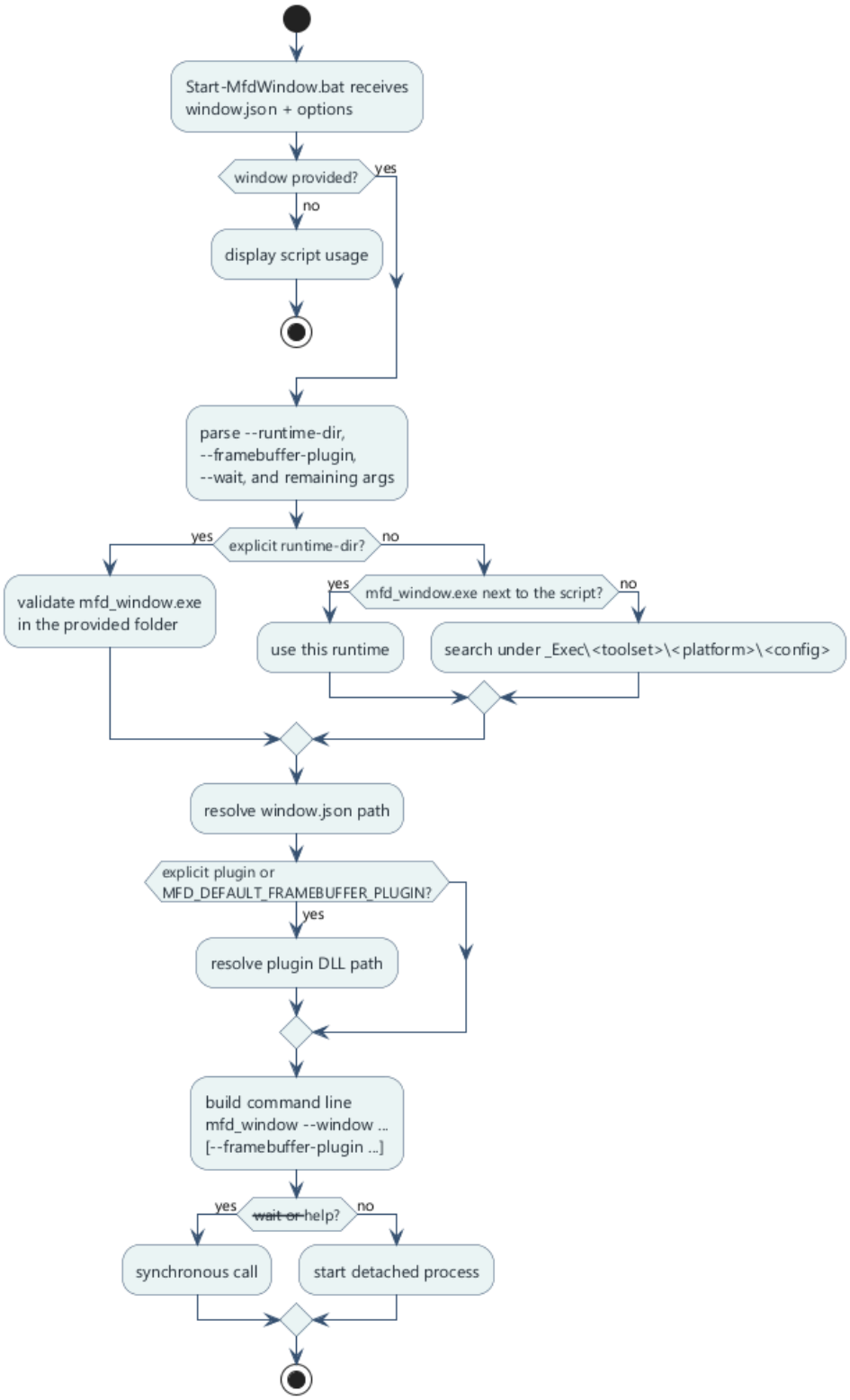
- compile as a DLL
- include `mfd/window/WindowLauncherPlugin.h`
- export `MfdGetWindowFramebufferPluginApi`
- keep the DLL boundary `noexcept`
- keep implementation detail in private C++ types behind `plugin_context`

7.14 Final integration checklist

Before wiring the plugin into a full runtime launch, verify all of the following:

1. the DLL exports `MfdGetWindowFramebufferPluginApi`
2. `info.abi_version` matches `MFD_WINDOW_FRAMEBUFFER_PLUGIN_ABI_VERSION`
3. `init`, `submit_frame`, `close`, and `destroy` are all populated
4. `submit_frame` validates the descriptor before touching pixels
5. pixels are copied or handed off before the callback returns
6. the async path does not block the render thread
7. the plugin can be exercised with a synthetic stack-allocated frame

8. Launch a window with a script and a framebuffer plugin



Launch script flow

The repository already ships a reusable batch launcher:

CODE TEXT

```
Scripts/Start-MfdWindow.bat
```

8.1 Use the launcher as the stable runtime entry point

Minimum call:

```
Start-MfdWindow.bat <window.json>
```

Full call shape:

```
Start-MfdWindow.bat <window.json> [--runtime-dir <dir>] [--framebuffer-plugin <plugin.dll>]
[--wait] [extra launcher args]
```

Use this script when you want one repeatable runtime entry point for authored windows, staged binaries, and optional framebuffer-plugin injection.

8.2 Supported arguments

Argument	Use
<window.json>	authored window to load
--runtime-dir <dir>	explicit folder that contains mfd_window.exe
--framebuffer-plugin <plugin.dll>	plugin DLL to inject into the runtime
--wait	keep the launcher attached to the child process
extra launcher args	forwarded to mfd_window

8.3 How the script resolves mfd_window.exe

Resolution order:

63. 1. the folder passed through `--runtime-dir`
64. 2. the script folder if `mfd_window.exe` is already there
65. 3. `_Exec/<toolset>/<platform>/<config>`
66. 4. a fallback scan under `_Exec`

This makes the same script usable from a staged delivery tree and from a developer build tree.

8.4 How plugin resolution works

The plugin path can come from:

- `--framebuffer-plugin`
- `MFD_DEFAULT_FRAMEBUFFER_PLUGIN`

The script then resolves the path relative to:

- the explicit absolute path if already absolute
- the runtime directory
- the repository root

- the script directory
- the current working directory

8.5 Example of a preconfigured project launcher

`Scripts/Start-MfdMinimal.bat` is intentionally small:

```bat
@echo off
setlocal
set "MFD_DEFAULT_FRAMEBUFFER_PLUGIN=mfd_framebuffer_stdout_plugin.dll"
call "%~dp0Start-MfdWindow.bat" "assets/windows/demo_pages_minimal.json" %*
exit /b %ERRORLEVEL%
```

This is a good pattern for project-specific launchers: set one default window, optionally set one default plugin, then delegate everything else to `Start-MfdWindow.bat`.

8.6 Example with an explicit custom plugin

```powershell
.\Scripts\Start-MfdWindow.bat `
  "assets/windows/demo_pages_minimal.json" `
  --framebuffer-plugin "build\vs2022-win32\examples\Debug\my_framebuffer_plugin.dll" `
  --wait
```

Equivalent direct runtime call:

```powershell
mfd_window --window assets/windows/demo_pages_minimal.json --framebuffer-plugin
my_framebuffer_plugin.dll
```

8.7 Why the script is still useful when the CLI exists

The script already solves three tedious runtime problems:

- finding the staged executable
- resolving the plugin path from several likely locations
- keeping a consistent project-local launch convention

That is why it remains the preferred operator entry point even when the underlying `mfd_window` CLI is available.

8.8 Write your own project launcher

```bat
@echo off
setlocal
set "MFD_DEFAULT_FRAMEBUFFER_PLUGIN=my_encoder_plugin.dll"
call "%~dp0Start-MfdWindow.bat" "assets/windows/my_window.json" %*
exit /b %ERRORLEVEL%
```

Keep the custom script small. Let the shared launcher own path resolution and argument parsing.

8.9 Launch checklist with a framebuffer plugin

Before launch, verify:

- the window JSON exists
- `mfd_window.exe` exists in one resolvable location
- the plugin DLL exists in one resolvable location
- the plugin exports `MfdGetWindowFramebufferPluginApi`
- the plugin ABI version matches the runtime

8.10 If the plugin does not load

Check in this order:

67. 1. incorrect DLL path
68. 2. missing DLL runtime dependencies
69. 3. missing exported entry point
70. 4. incomplete callback table
71. 5. invalid `struct_size` or `abi_version`
72. 6. crash or invalid return inside `init`

9. Overall recommended checklist

9.1 End-to-end project workflow

1. author the window and its pages in `mfd_editor`
2. save the authored assets in `assets/`
3. expose only the runtime surface the client really needs
4. declare dynamic template bindings and page strobe behavior intentionally
5. regenerate `Ui.h`, `Ui.cpp`, and `<window>.generated.map`
6. rebuild the client target that includes the generated `.cpp`
7. launch `mfd_window`
8. publish coherent batches through `CommandClient` or `LatestBatchPublisher`
9. poll feedback if page activity or strobe capture matters
10. attach a framebuffer plugin when the rendered image must leave the runtime

9.2 Design errors to avoid

- author directly in `_Exec`
- expose too many primitives without operational value
- keep stale generated outputs after an authored contract change
- scatter `SendBatch()` calls throughout the frame
- bypass runtime feedback when logic depends on authoritative page or capture state
- keep framebuffer pointers after `submit_frame` returns

9.3 Files every integrator should know

File	Why it matters
assets/windows/<window>.json	generator and runtime entry point
assets/windows/<window>.generated.map	transport ids and compatibility hash
mfd_client_api/generator/scripts/generate_ui.py	generation logic and --print-inputs behavior
mfd_client_api/generator/ClientApiGenerator.cmake	official CMake integration path
mfd_common_api/include/mfd/control/CommandClient.h	transport publication facade
mfd_client_api/include/mfd/client/Animation.h	generated-handle runtime behavior and feedback helpers

mfd_client_api/include/mfd/client/LatestBatchPublisher.h	latest-state asynchronous publisher
mfd_window_plugin_api/include/mfd/window/WindowLauncherPlugin.h	stable framebuffer-plugin ABI
Scripts/Start-MfdWindow.bat	runtime and plugin launch path

9.4 Final takeaway

The clean MFDStudio path is:

- author the visual structure in the editor
- generate one typed C++ contract from the authored window
- publish one coherent client batch per frame
- let runtime feedback confirm page activity and strobe capture
- copy framebuffer pixels safely before crossing into async plugin logic

This keeps the repository modular, the client surface intentional, and the runtime behavior debuggable.

Appendix A. Quick Reference

This appendix is the one-page cheat sheet for the API calls that appear most often in a production client.

API call	Signature	3-word use
activate page	bool ActivatePage(const GeneratedPage& page)	select active page
set page view	bool SetPageView(const GeneratedPage& page, Vec2 center, float zoom)	pan and zoom
access whole window	WindowDisplay& Window() noexcept	window display state
collect dirty commands	std::vector<mfd::UserCommand> BuildBatch()	gather dirty commands
build tracked batch	mfd::CommandBatch BuildCommandBatch(std::uint32_t sequence = 0)	attach hash sequence
send one frame	bool SendBatch(const mfd::CommandBatch& batch)	publish current frame
queue freshest frame	bool SubmitLatest(LatestBatchPublisher&, std::uint32_t sequence = 0)	stream latest state
poll runtime feedback	std::size_t PollFeedback(IExchangeChannel&, std::size_t maxMessages = 64, std::string* error = nullptr)	drain feedback packets
check page activity	bool IsActive() const noexcept	runtime page active
check capture state	bool IsStrobeCaptured() const noexcept	runtime target captured